

UNIVERSITÀ DI PISA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Laurea Triennale in Ingegneria Informatica

Studio dell'architettura e della programmazione dell'Elaboratore Elettronico Aritmetico (ELEA) 9003 dal punto di vista moderno

Relatore:

Prof. Giuseppe Lettieri

Candidato:

Nicholas Del Buono

ANNO ACCADEMICO 2025/2026

Indice

1 Un elaboratore General Purpose **5**
1.1 Lo scopo di Elea 6
1.2 Principi di modularità 8
1.3 La manutenzione oggi 10

2 Il ruolo del nucleo **13**
2.1 I Governi 13
2.1.1 Unità di Governo Centrale 14
2.1.2 Il Governo Unità a Nastro — GUN 15
2.2 Contemporaneità operativa 16
2.2.1 Lo scheduler 18
2.3 Eccezioni 19

3 L'Unità centrale **21**
3.1 Memoria Principale 22
3.1.1 Indirizzamento 24
3.2 Il registro Accumulatore 25
3.3 I registri T 27

4 Logica aritmetica **29**
4.1 La rappresentazione numerica 29
4.1.1 La codifica 30
4.2 Realizzazione dell'aritmetica 32
4.2.1 La gestione del segno 34

5 L'assembler Olivetti **37**
5.1 Il formato delle istruzioni 37
5.2 Organizzazione della programmazione 41
5.3 Programmi d'esempio 43
5.3.1 Divisione 43
5.3.2 Calcolo della media 45
5.3.3 Calcolo di una fattura 46

Considerazioni finali **49**

Capitolo 1

Un elaboratore General Purpose

Lo scopo di questo testo è offrire un nuovo punto di vista su Elea 9003 (*ELaboratore Elettronico Aritmetico*), che avvicini l'approccio nozionistico dei manuali dell'epoca a uno studio appropriato per uno studente di ingegneria d'oggi. Il mio focus si concentra su un approccio ad alto livello, tralasciando considerazioni che concernono l'elettronica e la realizzazione fisica dei meccanismi descritti, quando non necessario. Inoltre, propongo un insieme di programmi inediti che permettano di comprendere le potenzialità di questo elaboratore più a fondo.

L'analisi tecnica condotta in questo elaborato non si limita allo studio della manualistica originale che Olivetti distribuiva agli operatori, ma si basa inoltre sulla mia personale esperienza diretta sulla macchina. Attualmente, presso l'Istituto Tecnico "Enrico Fermi" di Bibbiena (AR), è conservato l'ultimo esemplare di Elea 9003 (esemplare 9003/02) ancora in condizioni di semi-operatività. Grazie alla collaborazione dei tecnici che mantengono la macchina, alla partecipazione attiva alle operazioni di manutenzione e al confronto diretto con tecnici storici Olivetti, è stato possibile integrare dati teorici con osservazioni pratiche sulle motivazioni di varie scelte implementative.

Sebbene l'analisi storiografica non sia l'obiettivo primario di questo testo, alcuni cenni storici sono necessari per comprendere la rilevanza di Elea in ambito internazionale. Progettata e sviluppata tra il 1957 e il 1959 dal Laboratorio di Ricerche Elettroniche Olivetti, sotto la guida dell'ingegner Mario Tchou, è uno dei primi elaboratori a transistor, un'innovazione per l'epoca. La visione di Adriano Olivetti fu determinante nel guidare l'azienda verso l'elettronica, partendo da una consolidata leadership nel settore delle macchine per scrivere e del calcolo meccanico.

Il progetto Elea condivide origini e personale con lo sviluppo di un altro calcolatore elettronico: la CEP (Calcolatrice Elettronica Pisana), progetto avviato dall'Università di Pisa nel 1955 e al quale Olivetti partecipò. Nel novembre dello stesso anno Olivetti insediò infatti a Barbaricina, alle porte di Pisa, il proprio Laboratorio

di Ricerche Elettroniche, diretto fin dall'inizio da Mario Tchou. La CEP è ad oggi conservata presso il Museo degli Strumenti per il Calcolo di Pisa.

Per un'analisi storiografica più approfondita del contesto storico ed aziendale, si rimanda alla bibliografia [1].

1.1 Lo scopo di Elea

Scommettere sull'elettronica non risponde a un solo obiettivo. Un elaboratore elettronico avrebbe difatti permesso di ridurre considerevolmente le tempistiche e le percentuali d'errore umano in tutti i settori chiamati a operare con grandi quantità di dati, posizionando l'azienda come diretta concorrente dei colossi americani del settore.

Produrre un elaboratore a programma memorizzato consente inoltre di svincolare l'hardware da una funzione predeterminata: si concretizza così il concetto di elaboratore *general purpose*, in cui è l'utilizzatore a definirne lo scopo semplicemente caricando software differenti in memoria. Non dovrebbe sorprendere inoltre che Elea possieda la caratteristica che definisce un'architettura di **von Neumann**: un'unica memoria di lavoro, che ospita indifferentemente istruzioni e dati, da cui discende la possibilità per un programma di modificare durante l'esecuzione le proprie istruzioni, che in memoria sono dati come tutti gli altri. A questa possibilità Elea affianca un meccanismo dedicato: la modifica automatica delle istruzioni tramite i registri T — che, per inciso, non altera l'istruzione in memoria — che verrà discussa in seguito.

Per citare il caso dell'esemplare 9003/02, esso viene acquistato dalla banca Monte dei Paschi di Siena, che lo impiegherà principalmente per la contabilità del personale — un impiego amministrativo interno, più che rivolto alla gestione della clientela — sfruttando la possibilità di mantenere considerevoli quantità di informazioni in una memoria persistente e facilmente accessibile dall'elaboratore, grazie alla memoria a nastri magnetici.

La flessibilità della macchina permise inoltre la sua adozione da parte di grandi realtà industriali: presentata nel 1959 alla Fiera Campionaria di Milano, nel luglio 1960 un primo esemplare fu donato al Ministero del Tesoro, mentre la prima consegna su base commerciale avvenne a settembre dello stesso anno, presso il gruppo tessile Marzotto di Valdagno. In totale, vennero prodotti e installati circa quaranta esemplari del modello 9003, destinati a importanti aziende ed enti italiani.

Può risultare difficile immaginare le dimensioni di Elea, viste le nette differenze con i computer moderni. Si parla di un macchinario dalle dimensioni imponenti, paragonabili a due intere stanze, nel quale l'operatore può — e per utilizzarlo deve a tutti gli effetti — camminare al suo interno.
Questo aspetto è fortemente caratterizzante per il calcolatore: il suo design permette agli operatori di comunicare e fare manutenzione mantenendo sempre il contatto visivo con il coordinatore seduto al tavolo di comando.

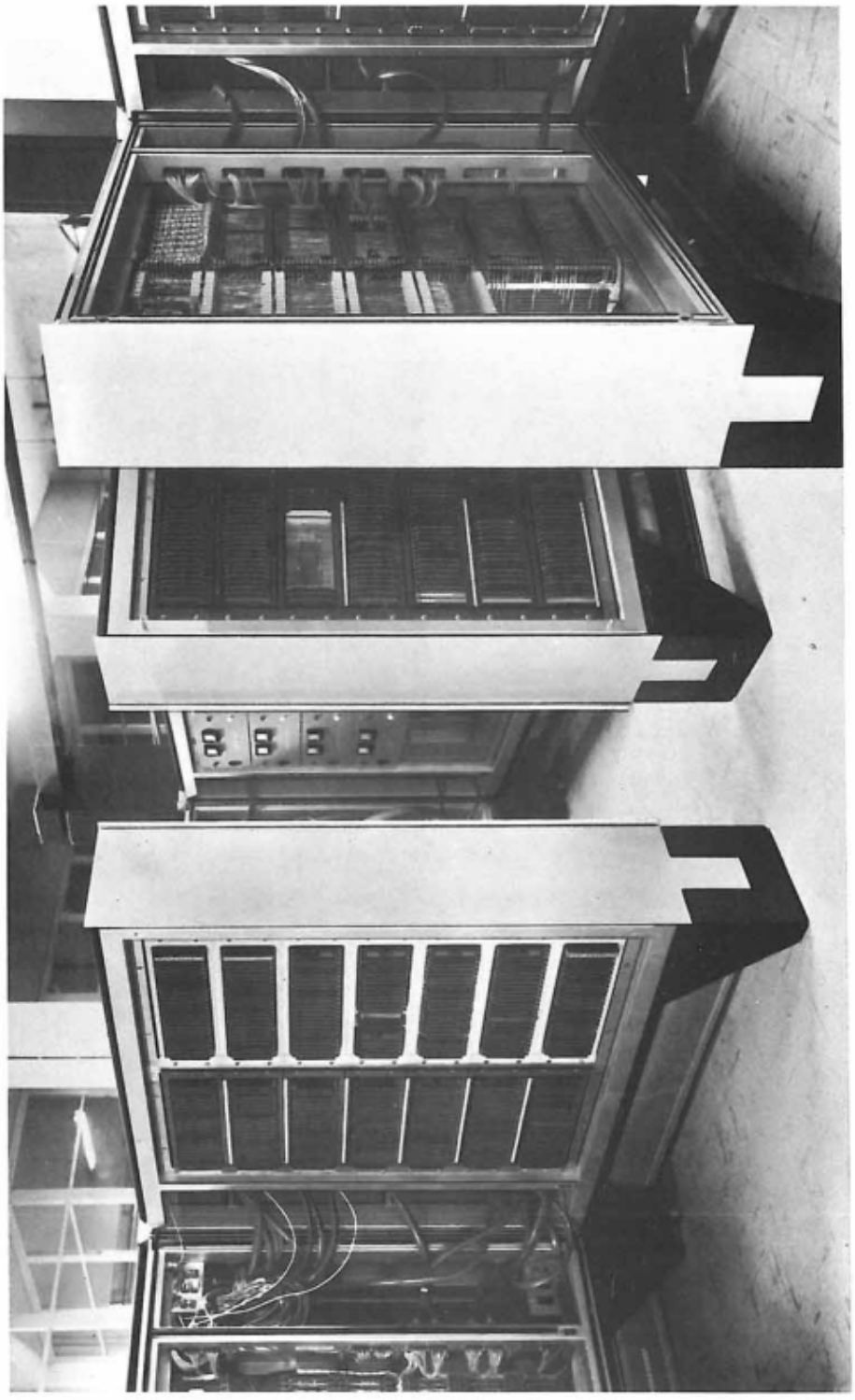


Figura 1.1: La struttura di Elea 9003. Fonte: [7]

Questa visione deriva dall'architetto e designer Ettore Sottsass, il quale progetta gli armadi, contenenti le componenti funzionali di Elea, di altezza contenuta (circa 160 cm) facendo passare i bus dati sopra la testa degli operatori. Per questo design Sottsass vincerà nel 1959 il *Compasso d'Oro*, premio per il design industriale.

Per un'analisi approfondita sul design di Elea 9003, si rimanda alla bibliografia [6].

1.2 Principi di modularità

Per accentuare il controllo esercitato dall'acquirente sull'elaboratore, Olivetti progetta Elea 9003 come un prodotto **espandibile** e **modulare** — "la struttura modulare, che caratterizza l'intero sistema, permette di adeguare la potenza dell'Elea al reale bisogno dell'utente"[1] e di collegare nuove unità con l'aumentare del volume di lavoro — concetto chiave anche dei calcolatori moderni.

In base alle necessità dell'utilizzatore era infatti possibile aumentare il numero delle unità a nastro magnetico (fino a un massimo di 20), gestite e attivabili tramite un apposito "governo", oppure ampliare la memoria principale — la moderna RAM — da 20.000 locazioni fino a un massimo di 160.000.

La stessa filosofia regola l'uso quotidiano della macchina, non solo la sua configurazione iniziale: nella parte bassa del quadro di controllo sono presenti 8 interruttori di alimentazione che permettono all'operatore di scegliere quali unità alimentare accendendo di volta in volta solo ciò che il lavoro in corso richiede. Per inciso, le unità che possono essere lasciate senza alimentazione, mantenendo la macchina operativa, sono le unità a nastro, il tamburo (non presente nell'esemplare 02) e i motori delle unità in linea — ovvero parte della componentistica d'input/output.

Un terzo livello di modularità riguarda infine la manutenibilità fisica dell'hardware. I circuiti che implementano tutta la logica dell'elaboratore, dalla "Unità aritmetica e logica" — il corrispondente della moderna ALU — fino ai "Governi" delle unità periferiche (i supporti hardware del sottosistema di I/O), sono di tipo standardizzato e realizzati su "piastrine" inseribili e facilmente sostituibili: una scelta che rende semplice ed economica sia la costruzione sia la manutenzione della macchina.

[1] Manuale Base di Programmazione, §1.2, pag. 4.

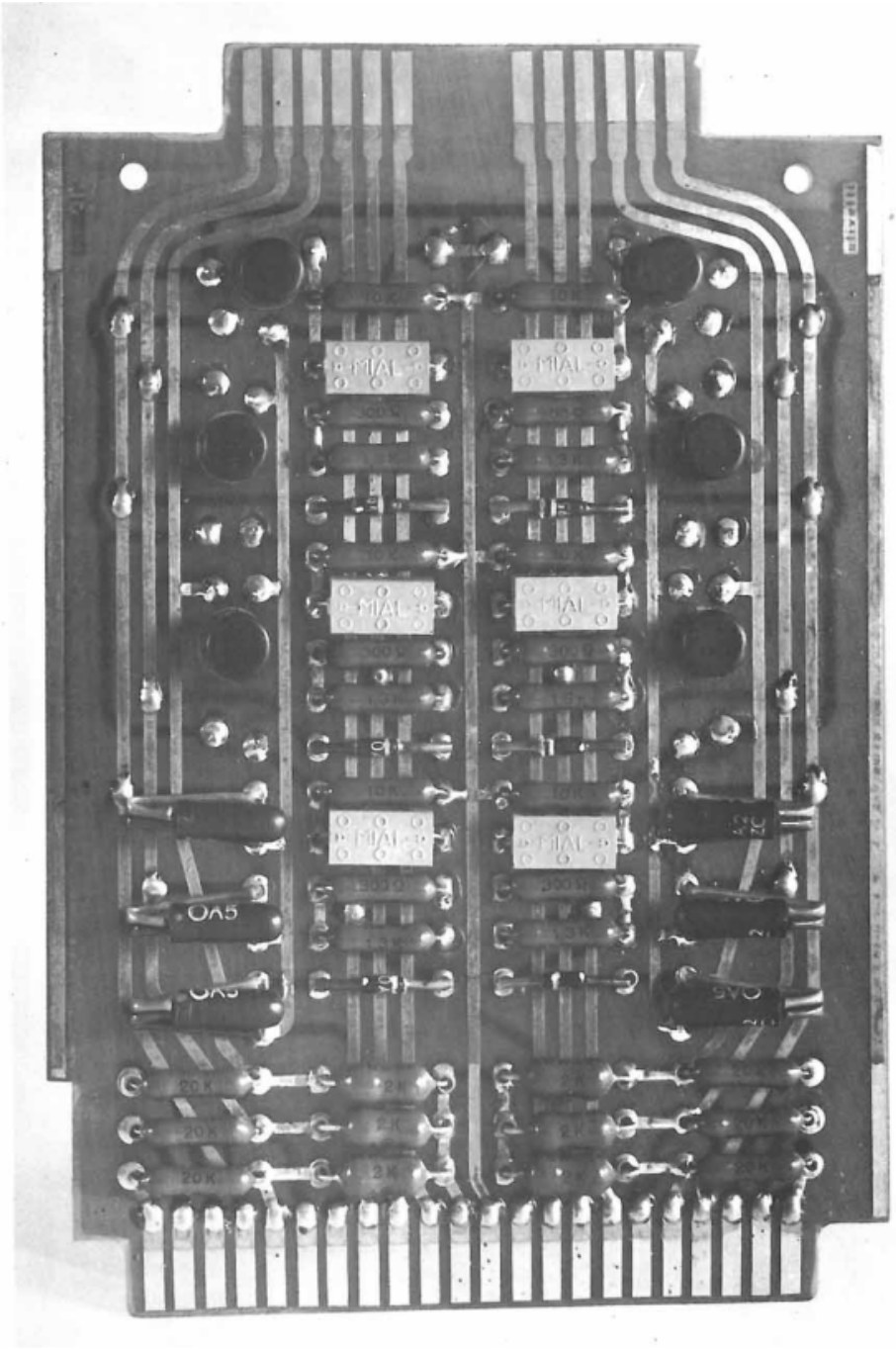


Figura 1.2: Un esempio di piastrina utilizzata nell'elaboratore. Fonte: [7]

1.3 La manutenzione oggi

Come già accennato, a Bibbiena, un piccolo comune in Casentino, è tutt'oggi presente l'esemplare numero 02 di Elea 9003, preservato nell'istituto tecnico Fermi. Ho avuto personalmente modo di partecipare alle opere di manutenzione con tecnici e ingegneri volontari facenti parte dell'associazione "Elea ETS" che si sono prefissati come obiettivo il restauro dell'ultimo esemplare completo esistente. Questa esperienza mi ha permesso di comprendere come all'epoca si potessero svolgere manovre di manutenzione di routine — sebbene molte delle tecniche che mi sono state presentate non siano oggi applicabili, a causa dello stato della consolle, il principale organo che permette di interfacciarsi con la macchina.

Il primo passo, quando si riscontra un problema, è individuarne la causa. Questo è possibile appunto tramite la consolle, che mediante delle spie permette di identificare un errore durante lo svolgimento di operazioni di routine, permettendo di effettuare del debugging — nel nostro caso — hardware.

Una volta ristretto dunque il campo, si ricade generalmente in tre principali casistiche: il guasto di un componente discreto, un range di tensione non rispettato o l'usura di un collegamento fisico. Nel caso in cui si debba sostituire un componente, esso può essere rimpiazzato — se presente — tramite uno dei ricambi forniti all'epoca (tuttora presenti in piccole quantità). Oppure tramite un componente elettronico moderno funzionalmente analogo. Molte volte ci si trova ad affrontare dei dubbi morali: più componenti originali vengono sostituiti da quelli moderni, più l'elaboratore perde il suo fascino. Purtroppo non sempre si trovano delle alternative, e le sostituzioni vengono effettuate nella speranza di riportare l'operatività dell'elaboratore il più vicino possibile a quella originale.

Sono inoltre presenti dei programmi di diagnostica, conservati in forma cartacea a Bibbiena, pensati per testare i singoli organi ed effettuare una manutenzione mirata — un'operazione che richiede però di poter nuovamente caricare ed eseguire programmi sull'elaboratore, cosa che non è attualmente possibile. Per questo motivo le opere di restauro si concentrano, ad oggi[2], nel ristabilire l'operatività della consolle, senza la quale nessun'altra funzionalità è utilizzabile — diagnostica compresa.

- *Manuale Base di Programmazione*: cap. 1 (pag. 1–10); cap. 2 (pag. 11–21); cap. 11 (pag. 231–245).
- *Introduzione al Sistema ELEA*: Parte E, cap. 2 (pag. 139–145), §3.1 (pag. 146) e §4.3 (pag. 153–155).
- *La memoria a nastri magnetici*: Parte A, cap. 1 (pag. 1–3).

[2]Ottobre 2026.

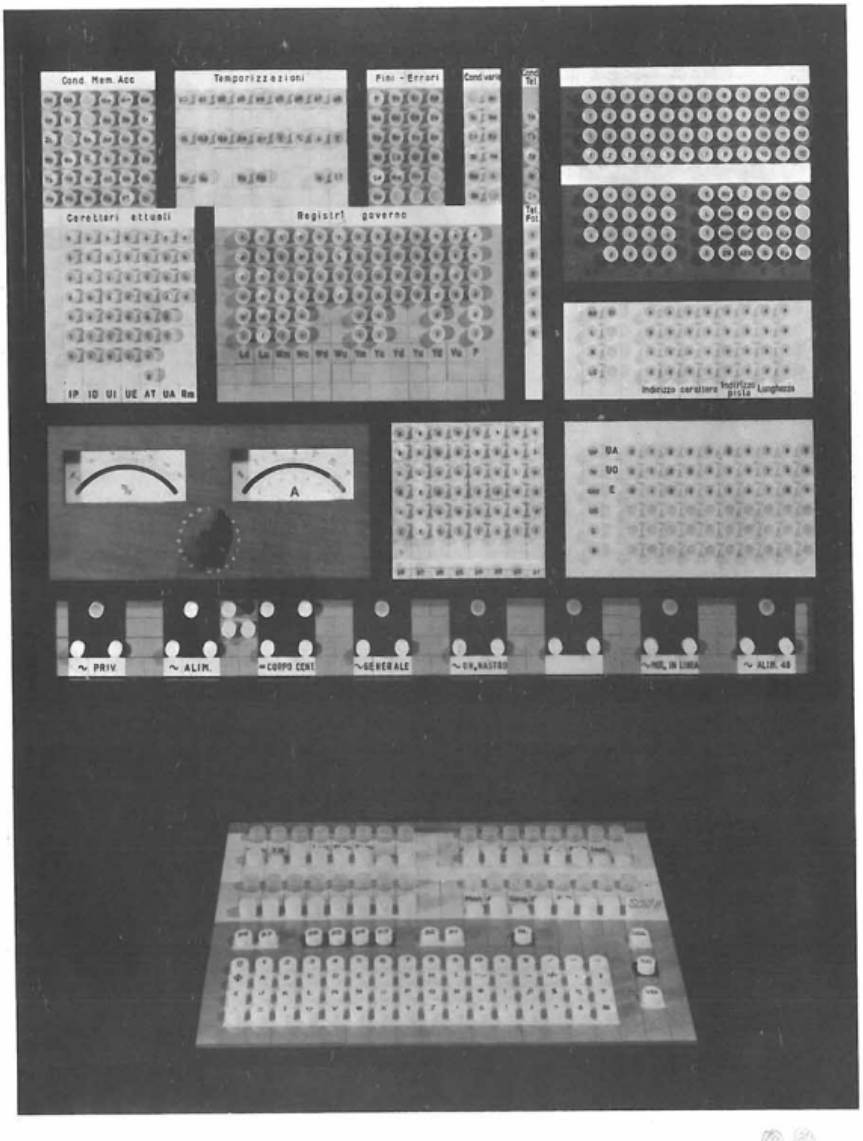


Figura 1.3: Il tavolo di comando (la consolle). Fonte: [7]

Capitolo 2

Il ruolo del nucleo

Parlare di nucleo, come parallelismo del concetto moderno del termine, non è certo affar semplice. Prima di iniziare a discutere di cosa possa esser considerato parte integrante del kernel di Elea, è necessario puntualizzare la totale assenza di un Sistema Operativo. Infatti, nonostante la presenza di memoria persistente a disposizione degli organi di governo e del programmatore, non furono implementate routine di sistema che permettessero ai programmi d'interfacciarsi con l'hardware.

Pur in assenza di un Sistema Operativo, le funzioni di un nucleo restavano indispensabili: nemmeno allora era pensabile di lasciare al programmatore la gestione diretta di ogni periferica, la rilevazione degli errori e la sincronizzazione fra organi. In Elea queste funzioni sono affidate interamente all'hardware: le periferiche sono gestite da appositi organi, i governi; gli errori sono rilevati da controlli automatici: perfino l'alternanza fra più programmi è regolata da uno scheduler a priorità fissa, interamente cablato. A mio avviso si tratta di una scelta più che di un limite dell'epoca: esistevano già alcuni esempi di software per la gestione dell'input/output, ma i progettisti riponevano maggiore fiducia nelle soluzioni fisiche.

2.1 I Governi

Appurato quindi che la gestione di tutti i compiti di un nucleo è affidata all'hardware, viene definita "Governo" una parte dell'architettura il cui scopo è gestire un sottoinsieme del sistema: l'unità centrale, le unità a nastro magnetico, ciascuna unità in linea.

Il principale è l'unità di governo centrale, che dirige l'intera elaborazione. Le unità a nastro magnetico fanno capo tutte al GUN, mentre ciascuna unità in linea — lettore o perforatore di schede, lettore di nastro perforato, stampante — dispone di un governo dedicato, con una piccola "memoria di transito" capace di contenere i caratteri di un "ciclo meccanico" di lavoro, come una riga di stampa o una scheda. Le unità in linea, a loro volta, non comunicano direttamente con l'unità centrale, ma attraverso un organo comune a tutte: il sincronizzatore.

Le unità fuori linea sono invece apparati esterni che operano su un supporto intermedio — nastri o schede — senza mai dialogare direttamente con l'elaboratore, non necessitando quindi di alcun coordinamento in tempo reale con esso.

La trattazione dei singoli governi delle unità in linea esula dagli scopi di questo testo.

2.1.1 Unità di Governo Centrale

Quest'unità rappresenta il blocco direttivo dell'elaboratore, paragonabile alla moderna *Control Unit* (CU): come nelle architetture odierne, non esegue essa stessa la funzione richiesta, ma orchestra gli organi che la eseguono. Una differenza riguarda però la gestione di più flussi di esecuzione: una Control Unit non conosce il concetto di "processo", il cambio di contesto è infatti un'operazione svolta dal Sistema Operativo. Il governo di Elea, non potendo contare su un livello software superiore, gestisce questa logica da solo: ad ogni transizione fra un'istruzione e la successiva, stabilisce quale delle sequenze attive abbia diritto di procedere. Le unità preposte all'esecuzione vera e propria delle istruzioni sono dislocate in armadi fisici distinti in prossimità dell'Unità di Governo, e insieme a essa questi moduli costituiscono l'Unità Centrale di Elea.

Tra le funzioni principali dell'Unità di Governo Centrale — per comprenderne meglio lo scopo — figurano:

- **Prelievo**: l'estrazione sequenziale dell'istruzione dalla memoria principale.
- **Interpretazione**: la lettura del carattere di funzione, che stabilisce il tipo di operazione da compiere. Esso viene memorizzato in un registro F, dal quale viene inviato ad un decodificatore combinatorio, che lo mappa nel segnale di attivazione relativo a tale istruzione.
- **Avvio**: l'attivazione degli organi fisici competenti per l'esecuzione.
- **Controllo**: la verifica del corretto svolgimento dell'operazione.

A questi si aggiunge una capacità più generale: il governo può tenere conto di segnalazioni provenienti dagli organi centrali e periferici per modificare il corso delle operazioni. È a questa capacità che si deve, in particolare, l'arbitraggio fra le sequenze di programma attive quando richiedono simultaneamente lo stesso organo — un meccanismo che, a mio avviso, non ha nulla a che vedere con le interruzioni asincrone dei processori moderni: il governo si limita a stabilire, a ogni transizione fra un'istruzione e la successiva, quale sequenza *già in corso* abbia diritto di procedere, secondo un ordine di priorità fisso e noto a priori — la fase esecutiva di un'istruzione non viene infatti mai interrotta per farne eseguire un'altra con precedenza superiore.

2.1.2 Il Governo Unità a Nastro — GUN

Il GUN rappresenta uno degli organi di governo più complessi di Elea, preposto ad interfacciarsi con gli armadi delle unità a nastro magnetico. Questo modulo richiede un'analisi più approfondita sia per la sua centralità nell'architettura — visibile anche dalle note lasciate dai tecnici sui manuali — sia per la capacità, notevole per l'epoca, delle unità che gestisce.

I compiti di questo governo ricadono nella gestione dello scambio dati fra la memoria principale e i nastri magnetici e nella gestione delle unità a nastro in autonomia rispetto al governo centrale. Per quanto riguarda lo scambio delle informazioni, all'avvio dell'esecuzione di un'istruzione di lettura o registrazione l'indirizzo di memoria viene caricato nell'indirizzatore `Y` del canale esterno; da quel momento i caratteri circolano tra nastro e memoria senza l'intervento di altre istruzioni, mentre l'unità centrale prosegue sul canale interno — per inciso, i canali "interno ed esterno" rappresentano la suddivisione fisica del bus dati.

Se i due canali richiedono la stessa matrice di memoria (la memoria è divisa in due matrici), è l'unità centrale a cedere la precedenza, fermandosi per un periodo di cifra (l'analogo del ciclo di clock). Questo accesso alla memoria autonomo e parallelo all'elaborazione rende il GUN paragonabile a un moderno controller DMA, con due differenze: l'indirizzatore utilizzato non appartiene al GUN ma all'unità centrale e la fine di un trasferimento non è segnalata da un'interruzione, ma è il programma ad attenderla. Come in un sistema moderno, invece, l'integrità dei dati — garantita non interagendo con la zona di memoria che il nastro sta utilizzando — è lasciata al rigore del programmatore.

Le funzioni principali del GUN possono essere riassunte in:

- **Gestione della meccanica**: il rispetto dei limiti fisici delle unità a nastro, ritardando i comandi finché queste non sono pronte a eseguirli.
- **Esecuzione autonoma**: lo svolgimento del riavvolgimento, spostamento di blocchi e copia fra nastri senza impegnare l'unità centrale.
- **Verifica**: la rilettura di ogni carattere appena registrato e il confronto con quello inviato.
- **Ricerca**: l'arresto automatico di una copia fra due nastri al primo blocco il cui inizio è maggiore o uguale a una parola chiave fornita dal programma.
- **Segnalazione**: la rilevazione di condizioni come unità occupata, fine del nastro o errore di registrazione, che il programma può interrogare con apposite istruzioni di salto.

Per quanto riguarda le unità a nastro, al GUN possono esserne collegate fino ad un massimo di 20. Di norma ne viene impiegata una sola per volta, in lettura o in registrazione; solo alcune istruzioni ne impiegano due contemporaneamente, una in lettura e una in registrazione.

La capacità di una singola bobina dipende dalla lunghezza media n dei blocchi, espressa in caratteri:

$$N = \frac{12.960.000 \cdot n}{n + 300}.$$

Con blocchi di 1000 caratteri, ad esempio, una bobina ne contiene 9.969.230. Esse sono inoltre facilmente sostituibili, così che si possano montare di volta in volta solo quelle attualmente necessarie.

2.2 Contemporaneità operativa

Per ottimizzare l'utilizzo dell'elaboratore, diminuendo le attese dovute alla gestione delle periferiche, Elea permette la gestione di più di un programma, alternando parti in interleaving a parti — dove possibile — in overlapping. Giusto per proporre una metrica di paragone, un periodo di cifra ha una durata di 10 μs, una scheda perforata viene letta in 120 ms — 12.000 periodi di cifra — e la stampante stampa una riga in 200 ms — 20.000 periodi di cifra. Ci si rende facilmente conto che la differenza in termini di ordini di grandezza non è trascurabile.

Il sistema Elea è in grado di gestire fino a tre sequenze di programma, ognuna con il proprio instruction pointer: l'elaborazione inizia sempre dal 1° programma, il principale, pensato per l'elaborazione dei dati e le operazioni aritmetiche — in una parola, per l'utilizzo dell'unità centrale. Da esso tramite le istruzioni `S2P` e `S3P` è possibile avviare rispettivamente il 2° programma, pensato per la gestione del nastro magnetico, e il 3° programma, denominato anche POL (Programma On Line) per la gestione appunto delle unità in linea — per inciso, il 3° programma può essere avviato anche dal 2°.

Quando più di una sequenza è attiva in un dato momento, l'unità di governo preleva un'istruzione alla volta tramite l'indirizzatore `W`, comune a tutti i programmi, secondo il sistema di priorità proposto nel seguito. Questo implica che il prelievo delle istruzioni viene eseguito in interleaving, mentre solo la fase esecutiva delle istruzioni in determinate circostanze può sovrapporsi.

Se una delle istruzioni `S2P` o `S3P` viene letta sul 1° programma mentre la sequenza che dovrebbe avviare è ancora in corso, il salto non viene effettuato e l'istruzione viene "riciclata": al suo termine l'indirizzatore `W` non avanza all'istruzione successiva, e la macchina rilegge ed elabora di nuovo la stessa istruzione. Si tratta quindi di un'attesa attiva, realizzata però dall'hardware anziché da un ciclo di istruzioni: il 1° programma ripete la `S2P` nei momenti in cui il 2° non può procedere, e quando quest'ultimo raggiunge il proprio `STOP` la `S2P` viene riciclata un'ultima volta ed effettua il salto, riavviandolo. Oltre che per l'avvio, `S2P` e `S3P` servono dunque come punto di sincronizzazione, ad esempio per attendere che le operazioni su nastro siano concluse prima di elaborarne i dati.



Figura 2.1: Alcune unità a nastro magnetico. Fonte: [7]

Overlapping

Per comprendere in quali fasi tali programmi possano eseguire in parallelo, è necessario esplicitare quali sono i fondamenti dell'hardware che favoriscono la contemporaneità: la suddivisione fisica del bus in canale interno ed esterno, la presenza degli indirizzatori `W` e `Y`, e la gestione relativa alle matrici pari e dispari della memoria di lavoro — componentistica descritta nel capitolo seguente.

Brevemente, ciascun canale dispone del proprio indirizzatore, `W` per quello interno e `Y` per quello esterno, in grado di raggiungere qualsiasi cella della memoria; la presenza di una matrice per gli indirizzi pari e una per gli indirizzi dispari permette ai due canali di lavorare su matrici diverse in ogni periodo di cifra, e quindi di leggere o scrivere due caratteri contemporaneamente. Il canale interno e il canale esterno possono dunque essere utilizzati in parallelo: poiché la fase preparatoria di ogni istruzione impegna sempre quello interno, la sovrapposizione riguarda le fasi esecutive. Se le fasi esecutive di due istruzioni richiedono allo stesso tempo due canali diversi, verranno eseguite contemporaneamente, altrimenti sequenzialmente. Fanno eccezione le istruzioni "doppie", che impegnano entrambi i canali — come le operazioni memoria-memoria — e bloccano l'altro programma fino al loro termine.

Per permettere al programmatore di sfruttare queste possibilità di parallelismo senza dover scrivere a mano un codice ottimizzato — che richiederebbe di tener conto della durata di ogni singola istruzione — sono state proposte tali sequenze di programma distinte, lasciando questo onere all'unità di governo.

2.2.1 Lo scheduler

Il meccanismo della priorità entra in gioco solo quando due programmi richiedono simultaneamente l'intervento dello stesso organo. Come principio si cerca di dare la precedenza alle istruzioni che riguardano organi lenti, che occupano l'unità centrale solo per piccoli intervalli di tempo, e lasciare che l'esecuzione di istruzioni più veloci operi in parallelo al loro lavoro. Per quanto riguarda il canale interno, si preferisce dare la precedenza al programma che lo impiega di meno.

Da queste premesse consegue un ordine di priorità fisso, ed è compito del programmatore collocare le istruzioni nel programma adatto per sfruttarlo. Il POL, che utilizza il canale interno solo per brevi periodi, ha quindi la priorità più alta; seguono il 2° programma, che lavora principalmente con il canale esterno, e infine il 1°, che impiega il canale interno sia nella fase preparatoria sia in quella esecutiva.

Si noti che la ripartizione dei compiti vista all'inizio di questa sezione è in gran parte una raccomandazione: tutti e tre i programmi possono contenere istruzioni di qualsiasi tipo, ma le istruzioni di comando delle unità in linea devono essere raccolte obbligatoriamente sul 3° programma.

Un'analisi dettagliata riguardo l'utilizzo dell'istruzione `SUO`, per la gestione della comunicazione tra il 3° programma e il sincronizzatore delle unità in linea, esula dagli scopi di questo testo.

La memoria t

Il passaggio da una sequenza di programma a un'altra dovrebbe essere il più possibile trasparente all'esecuzione del singolo: a ogni cambio di "contesto" sarebbe quindi necessario conservare lo stato che il programma interrotto lascia nei registri della macchina, per ritrovarlo intatto alla ripresa. Elea rispetta questo principio solo in parte.

Questa trasparenza è gestita solo per quanto riguarda i 5 flip-flop di stato, paragonabili a un registro dei flag. Essi sono consultati dalle istruzioni di salto condizionato e risiedono in un'unica risorsa fisica condivisa da tutti e tre i programmi: in un dato istante devono riflettere solo lo stato di quello attualmente in esecuzione, altrimenti un confronto eseguito dal 2° programma potrebbe alterare l'esito del salto condizionato che il 1° si prepara a eseguire.

La memoria t, composta da tre celle (una per programma), salva i flag nella cella del programma "interrotto" e li ripristina da quella del programma che riprende l'esecuzione. La differenza rispetto a un cambio di contesto moderno è la stessa già osservata per l'unità di governo: qui il salvataggio e il ripristino sono cablati nella fase preparatoria delle istruzioni, non gestiti da un livello software esterno.

Il resto dello stato, invece, non viene salvato, coerentemente con la ripartizione dei compiti raccomandata, che affida le operazioni aritmetiche al 1° programma: per accumulatore e registri T il manuale consiglia di non usare gli stessi registri in due programmi e di riservare l'accumulatore a uno solo, oppure di "scaricarli prima e ricaricarli dopo averli usati"[1].

2.3 Eccezioni

Elea non dispone di meccanismi di interruzione come quelli degli elaboratori moderni, ma deve comunque poter gestire gli errori dovuti a un malfunzionamento della macchina, come ad esempio un bit alterato durante un trasferimento. Tale compito si articola nelle seguenti fasi: l'hardware rileva l'errore, lo registra in appositi organi del governo e ne segnala l'ubicazione agli indicatori del tavolo di comando. È poi il programma a interrogare questi organi tramite apposite istruzioni di salto su errore — una per ciascun organo su cui si effettuano tali controlli, più l'istruzione `SE`, che salta in presenza di un errore qualsiasi — e a ricorrere, se necessario, a un programma di correzione. Come per la fine di un trasferimento su nastro, nulla interrompe il programma: è il programmatore a dover prevedere dove e quando gestire tali controlli.

Gli errori vengono rilevati da due meccanismi distinti, con ambiti di applicazione diversi: il primo agisce su ogni scambio di informazioni fra gli organi, il secondo è specifico dell'unità aritmetica e logica.

Il **controllo di disparità** si basa su un settimo bit, aggiunto ai sei di ogni carattere e scelto in modo che il numero di bit a 1 sia sempre dispari: l'alterazione

[1] Manuale Base di Programmazione, §9.3.2, pag. 188

di un singolo bit rende il conteggio pari, venendo quindi rilevata. Tale controllo si applica ai caratteri in memoria e a tutti i trasferimenti all'interno dell'elaboratore. Le unità a nastro vi affiancano una verifica specifica della registrazione: rileggono quanto appena scritto e lo confrontano con il dato originale. In caso di differenza la registrazione prosegue, ma l'errore resta disponibile per l'istruzione `SEN`.

Il secondo meccanismo è la **prova del 3**, riservata alla verifica dell'unità aritmetica e logica. Si basa sul principio per cui il resto della divisione per 3 del risultato di un'operazione aritmetica coincide con il resto della divisione per 3 del risultato della stessa operazione eseguita sui resti degli operandi: mentre l'operazione procede, un circuito dedicato calcola entrambi i valori e, se differiscono, segnala l'errore. È significativo, anche alla luce dell'assenza dell'istruzione di divisione discussa nel capitolo 4, che questo resto non venga calcolato eseguendo davvero una divisione, ma tramite un semplice contatore ciclico a tre posizioni.

- *Manuale Base di Programmazione*: cap. 2 (pag. 11–21); cap. 3 (pag. 23–32); §4.1 (pag. 33); cap. 5 (pag. 53–58); §7.10 (pag. 151–155); capp. 8–10 (pag. 157–230).
- *Introduzione al Sistema ELEA*: Parte E, capp. 4–5 (pag. 150–159) e capp. 8–10 (pag. 166–170).
- *Descrizione logica degli organi*: capp. 3–4 (pag. 14–27); cap. 6 (pag. 38–39); cap. 9 (pag. 94–99); cap. 14 (pag. 124–149).
- *Descrizione logico funzionale*: capp. 15–17 (pag. 109–135); capp. 19–23 (pag. 153–208).
- *La memoria a nastri magnetici*: Parte A (pag. 1–27).

Capitolo 3

L'Unità centrale

Per fare una trattazione dei singoli organi costituenti il sistema, ritengo opportuno chiarire a cosa ci si riferisce con il termine "Unità Centrale" in Elea 9003. Esso descrive l'insieme di tutte le componenti principali che permettono l'elaborazione dell'informazione, composto da: Unità di Governo, memoria di lavoro, registri T, Accumulatore, "Unità Aritmetica e Logica" ed il tavolo di comando.

Una visione d'insieme

Tutti gli organi precedentemente citati sono interconnessi tramite due canali — bus — fisicamente distinti che attraversano l'intera Unità Centrale, ciascuno con un compito specifico.

Il *canale esterno* collega la memoria principale al governo delle unità a nastro magnetico (GUN) e, se presente, a quello del tamburo magnetico (GUT) — assente nell'esemplare di Bibbiena — sia in lettura che in registrazione; viene inoltre impiegato nelle operazioni fra due zone di memoria, per far giungere uno dei due operandi. Il *canale interno* collega invece la memoria agli organi detti appunto "interni": l'Unità Aritmetica e Logica, l'accumulatore e i registri T, il governo del calcolatore e il sincronizzatore, attraverso il quale transitano anche le informazioni delle unità in linea.

Analizzando le componenti principali di questo blocco, emergono similitudini e differenze significative rispetto ai calcolatori moderni. Un esempio è la presenza del registro Accumulatore: a prima vista la sua funzione — un'area d'appoggio distinta dalla memoria principale — richiama il concetto di memoria cache. L'analogia però non regge: l'Accumulatore e la memoria principale innanzitutto richiedono gli stessi tempi d'accesso, essendo costruiti con la stessa tecnologia, i nuclei di ferrite. Entrambi sfruttano infatti $10 + l$ periodi di cifra, dove l è la lunghezza dell'operando, per le istruzioni che utilizzano tali organi. Oltre a non offrire alcun vantaggio in termini di tempistiche, l'Accumulatore non è nemmeno trasparente al programmatore come lo sarebbe una cache — è quest'ultimo, anzi, a doverne gestire esplicitamente il contenuto.

La sua struttura a puntatore (DA) e il bit di fine (gA), descritta nel dettaglio più avanti, permette di mantenere più risultati intermedi contemporaneamente durante un calcolo articolato in più passaggi. Inoltre, è la destinazione dei risultati dell'operazione di moltiplicazione, il che spinge il programmatore al suo utilizzo nonostante operazioni più semplici come la somma possano comprendere unicamente la memoria principale (`PUM`+`MEM`/`±MM`/`CMM`).

Emerge un vantaggio ulteriore, diretta conseguenza della distinzione fra canale interno ed esterno appena introdotta e della contemporaneità operativa vista nel capitolo 2. Le operazioni che coinvolgono trasferimenti memoria-accumulatore occupano solo il canale interno, mentre quelle memoria-memoria occupano anche quello esterno, nello stesso periodo di cifra. Un'istruzione che usa solo il canale interno può quindi essere parallelizzata ad una che utilizza solo il canale esterno — ad esempio un trasferimento verso l'Accumulatore mentre è in corso una scrittura su nastro — cosa non possibile per un'istruzione memoria-memoria, che occuperebbe entrambi i canali e competerebbe quindi con qualsiasi operazione sul canale esterno già in corso.

Nel sistema Elea, come negli elaboratori moderni, il mantenimento in memoria dell'informazione si divide in vari livelli di persistenza e frequenza di utilizzo temporale: si parte dalle unità a nastro magnetico, dove risiedono i dati persistenti all'uso, per arrivare a supporti tanto più veloci quanto più frequentemente un dato deve essere richiamato. La memoria di lavoro quindi deve essere costruita con tecnologie che la rendano la più paragonabile possibile — a livello di tempistiche — alla velocità in cui vengono processate le istruzioni dall'elaboratore. Chiamata "Memoria principale", anche Elea naturalmente ha quindi un corrispondente della RAM.

3.1 Memoria Principale

Costituita da due matrici di nuclei di ferrite fisicamente indipendenti, una per gli **indirizzi pari** (Mp) e una per gli **indirizzi dispari** (Md), è l'unità di memoria in cui risiederanno i programmi durante il ciclo di vita dell'elaboratore. Ciascuno dei canali precedentemente trattati possiede un proprio registro indirizzatore — **W** per il canale interno, **Y** per il canale esterno — e ciascuno dei due registri può selezionare liberamente un indirizzo su *entrambe* le matrici.

Selezionando un indirizzo su ciascuna matrice, la macchina può leggere o scrivere due caratteri nello stesso periodo di cifra: uno destinato agli organi interni, attraverso il canale interno, e uno destinato agli organi esterni, attraverso il canale esterno. È questo doppio accesso a permettere a due programmi di procedere contemporaneamente, come discusso nel capitolo 2.

Perché ciò sia possibile, i due canali devono lavorare su matrici diverse: se il canale esterno è collegato alla matrice dispari, quello interno è collegato alla pari, e viceversa. Questo collegamento è gestito dai flip-flop `Zc` e `Zr`, e si scambia a ogni periodo di cifra. Se invece i due indirizzatori puntano a indirizzi della stessa parità, e quindi alla stessa matrice, gli organi interni vengono fermati per un periodo di

cifra, così che l'indirizzo del canale interno cambi parità rispetto a quello del canale esterno.

È significativo anche un altro meccanismo che sfrutta questa proprietà: è possibile collegare entrambi i canali a organi interni, permettendo con una sola istruzione operazioni che riguardano due operandi residenti entrambi in memoria — meccanismo su cui si basano le istruzioni `PUM+MEM/±MM/CMM`. Analogamente, è possibile collegare entrambi i canali a organi esterni, per trascrivere con una sola istruzione informazioni da un nastro a un altro, passando per la memoria, senza mai coinvolgere l'Unità Aritmetica e Logica.

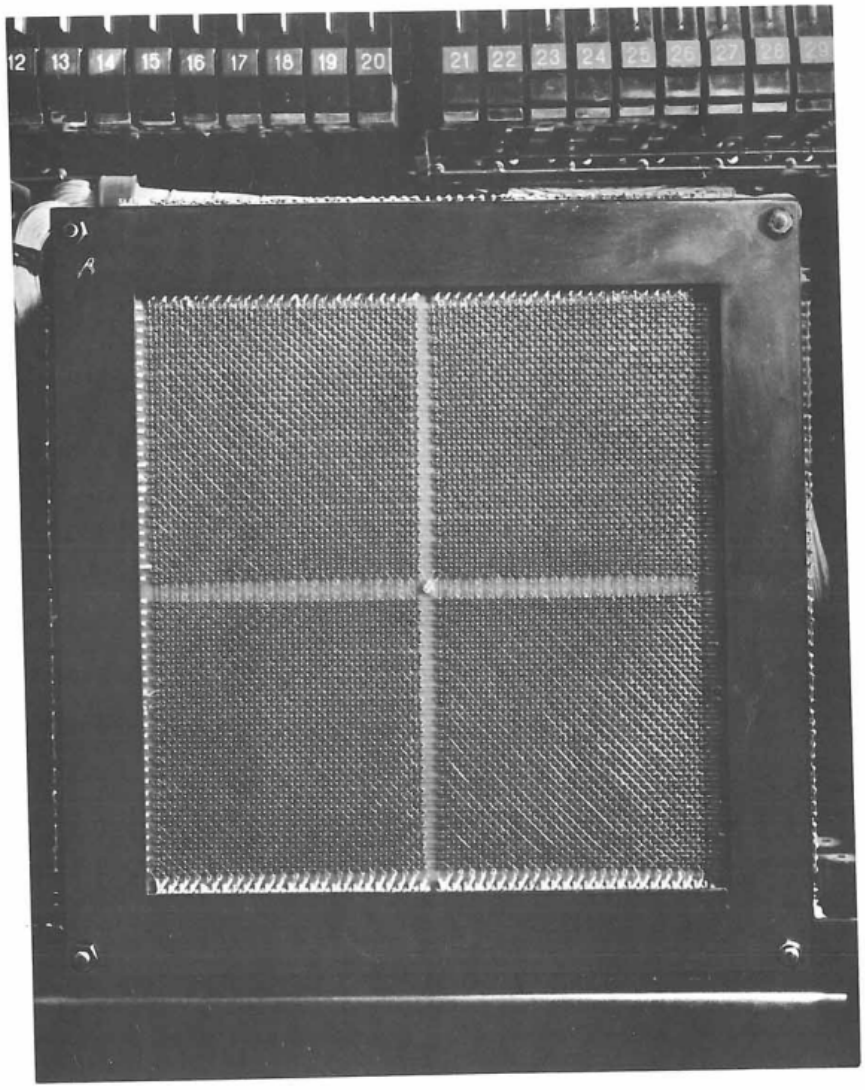


Figura 3.1: La memoria a nuclei di ferrite. Fonte: [7]

Struttura fisica

Viste le caratteristiche che permettono l'accesso dei due canali, posso analizzare le caratteristiche fisiche dal punto di vista della capacità e dell'indirizzamento.

La memoria è fisicamente divisa in elementi, ognuno composto da sette piani da 10.000 nuclei ciascuno, attraversati da sottili fili di rame intrecciati a mano: un blocco da 70.000 bit permette la rappresentazione di 10.000 caratteri alfanumerici. Perché sette piani, cioè sette bit per carattere? Perché nel sistema Elea per rappresentare tutto l'insieme dei caratteri (fig. 4.1) sono sufficienti 6 bit, con l'aggiunta di un bit per il controllo di disparità. Ogni locazione di memoria indirizzabile ha quindi il proprio bit di controllo dedicato: un errore che, per quanto improbabile, doveva comunque poter essere rilevato.

Questi sette piani formano quello che il manuale chiama un "elemento tecnologico" di memoria (10.000 caratteri); due elementi costituiscono la minima unità funzionale (20.000 caratteri), e un calcolatore può disporne da 1 a 8, per una capacità compresa tra 20.000 e 160.000 caratteri.

Un'ultima proprietà lega questa struttura fisica alla tecnologia sottostante: la memoria a nuclei di ferrite è caratterizzata da una lettura **distruttiva** — leggere una posizione ne cancella il contenuto. Se il dato dev'essere persistente ad una sua lettura, va quindi riscritto entro lo stesso periodo di cifra, operazione eseguita tramite un comando di rigenerazione dedicato (`CMdR` per la matrice dispari, `CMpR` per la pari), svolto dall'elaboratore in autonomia. Un effetto analogo, ma voluto dal programmatore, si ottiene con le istruzioni di trasferimento *con azzeramento*: `AoM` per l'accumulatore e `ToM` per i registri T, che dopo il trasferimento azzerano le posizioni trasferite.

3.1.1 Indirizzamento

L'esemplare 9003/02 presente a Bibbiena è equipaggiato con il minimo numero di locazioni sufficienti al funzionamento, 20.000. Questa caratteristica purtroppo non mi ha permesso di avere dei resoconti, da parte dei tecnici manutentori, riguardo le logiche di indirizzamento esteso, che meritano comunque una riflessione.

L'Indirizzamento della memoria è tanto affascinante quanto limitante, almeno per programmi operanti moli di dati considerevoli o composti da sequenze di codice molto lunghe, improbabili per l'epoca. La memoria, infatti, è circolare ed indirizzabile mediante indirizzi a quattro simboli modulo 40.000, tramite l'ausilio di configurazioni che non si limitano alle cifre. Ciò comporta che per configurazioni con più di 2 unità di memoria, non è possibile per programmi residenti in una coppia d'unità di interferire — tramite operazioni aritmetiche — con programmi residenti in un'altra. È però possibile esplicitare in quale modulo di memoria si sta operando, agendo direttamente sul secondo carattere da sinistra dell'indirizzo, quello delle centinaia: sono infatti i suoi due bit speciali (e, f) — la cui rappresentazione completa, insieme a quella degli altri quattro bit che compongono un carattere, sarà oggetto del

capitolo 4 — a selezionare, in base alla loro combinazione, uno dei quattro moduli fisici indipendenti.

È infatti possibile effettuare, ad esempio, `'I99 + 0001 = 0ϕ00`, che corrisponde a $79.999 + 1 = 40.000$ per effetto dell'aritmetica modulare applicata al calcolo degli indirizzi. Si noti che le migliaia e le centinaia seguono cicli distinti: passando da 79.999 a 40.000 le migliaia tornano a un carattere numerico "semplice" (`0`), mentre le centinaia restano "speciali" (ϕ), poiché l'indirizzo resta comunque nel secondo dei quattro moduli.

Isolamento

Si può osservare che questa aritmetica circolare produce, come effetto collaterale, un rudimentale isolamento fra programmi residenti in moduli diversi: un indirizzo calcolato per incremento non può mai sconfinare oltre il proprio modulo. È bene precisare che non si tratta di una protezione in senso proprio: nulla impedisce a un'istruzione di indicare esplicitamente un indirizzo al di fuori del proprio modulo tramite il carattere delle centinaia — l'isolamento vale solo per gli sconfinamenti prodotti aritmeticamente.

Olivetti non fornì meccanismi di protezione nativi per Elea verosimilmente perché non pertinenti alle esigenze dell'epoca: la macchina operava infatti in un contesto mono-utente e fortemente deterministico, supervisionato dall'operatore umano alla consolle di comando. Non dovendo eseguire codice di terze parti non verificato, il controllo dell'integrità della memoria era interamente affidato al rigore logico del programmatore.

3.2 Il registro Accumulatore

Memoria circolare costituita da 100 posizioni, l'Accumulatore permette di conservare risultati intermedi e di operarvi senza doverli trasferire in memoria principale. La logica d'accesso si basa su due supporti hardware: il registro DA e il bit gA. L'indirizzo iniziale di una parola contenuta nell'accumulatore è salvato nel registro DA, che agisce quindi da puntatore. La fine del dato, invece, è segnalata dalla presenza del bit gA, posto a 1 nell'ultima locazione occupata dal dato stesso — tale bit è presente in ogni locazione, materializzato da un ottavo piano di nuclei che si aggiunge ai sette usati per la registrazione dei caratteri e per la verifica di disparità.

Questa organizzazione permette di sfruttare l'Accumulatore per mantenere contemporaneamente in memoria vari risultati intermedi: per ciascuno di essi verrà settato il bit gA in corrispondenza del suo termine, mentre il registro DA funge da indice della parola su cui si sta operando in un dato momento.

Segue un esempio fornito da Olivetti per mostrare l'evoluzione del contenuto dell'accumulatore dopo una serie di operazioni. Per inciso: come già accennato, `AoM` è un'istruzione che trasferisce l'informazione dall'accumulatore alla memoria

principale con l'azzeramento delle posizioni trasferite. Un'analisi dettagliata sulla gestione del bit gA oltre quella già presentata esula dagli scopi di questo testo [7].

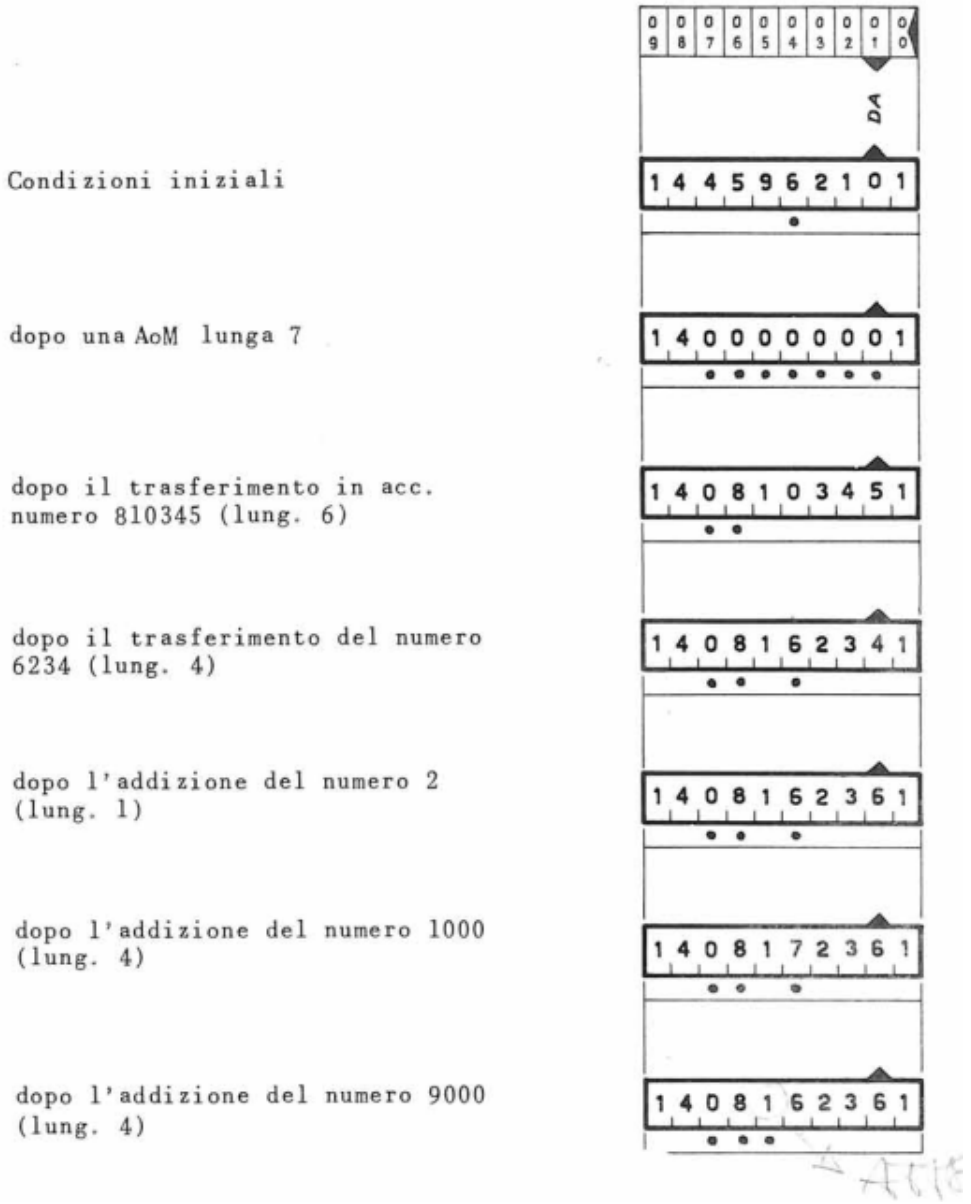


Figura 3.2: Un esempio d'utilizzo dell'accumulatore. Fonte: [7]

3.3 I registri T

I registri T sono, in analogia con i calcolatori odierni, un'unità di memoria d'appoggio utile al programmatore per facilitare la stesura dei programmi. Fisicamente costruiti anch'essi con nuclei magnetici di ferrite, hanno in totale una capacità di 200 caratteri indirizzabili di 5 in 5 tramite 40 registri.

Ogni registro ha una lunghezza massima di 10 caratteri — con l'eccezione di T0, che può contenerne fino a 100 ed è normalmente utilizzato per il moltiplicatore — e registri adiacenti si sovrappongono quindi per metà: ad esempio, la parte alta del registro TC coincide con la parte bassa del T4, e la sua parte bassa con la parte alta del T3 (fig. 3.3). Lo stesso principio — riutilizzare fisicamente lo stesso spazio di memoria sotto nomi diversi — è alla base dell'aliasing dei registri x86 (`%EAX`, che contiene `%AX`, a sua volta scomponibile in `%AH`/`%AL`), con una differenza di struttura: in x86 i registri sono annidati l'uno nell'altro, in Elea sono sfalsati di cinque posizioni.

Le due scelte nascono inoltre da esigenze diverse. In x86 l'aliasing è introdotto per garantire la retro-compatibilità, che permette al codice scritto per processori più vecchi e con registri più corti di continuare a funzionare; in Elea è invece un meccanismo verosimilmente dettato dal risparmio, che offre 40 registri al costo di 20. La sovrapposizione, del resto, non crea inconvenienti nell'uso più frequente dei registri T, quello di modificatori: in questo ruolo ne vengono infatti utilizzati al massimo i primi 5 caratteri, che non si sovrappongono mai fra registri diversi.

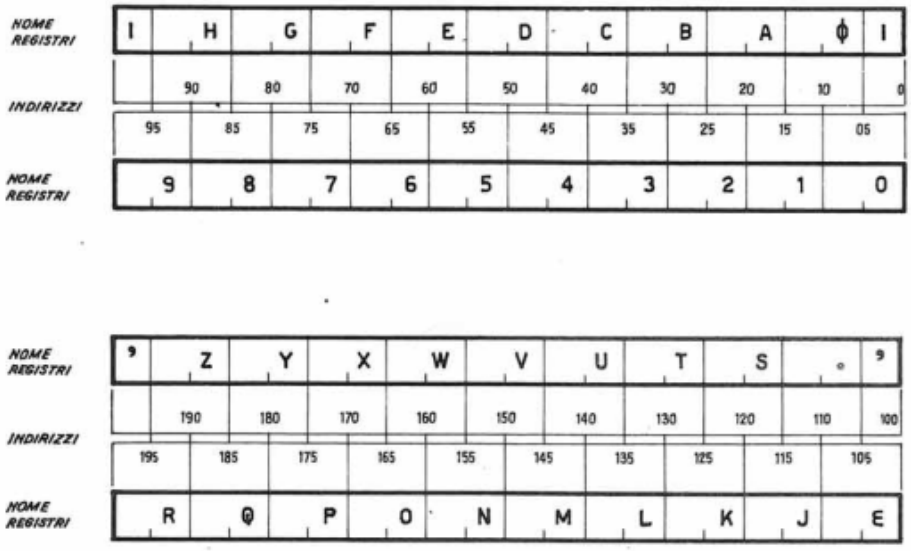


Figura 3.3: La sovrapposizione dei registri T. Fonte: [7]

Per questo organo di memoria non si ha un analogo del registro DA dell'accumulatore, visto che per ogni registro la locazione di partenza è la sua prima, ma è presente il bit gT, analogo del gA. Esso, presente in ogni locazione, indica la fine dell'informazione del registro riferito a tale indirizzo; anch'essi come i gA sono costruiti

tramite un ottavo piano di nuclei in parallelo a quelli inerenti al mantenimento delle informazioni.

Un particolare utilizzo dei registri T risiede nella funzionalità di modifica a tempo di esecuzione delle istruzioni. Per molte istruzioni è infatti possibile specificare un registro T detto "modificatore" il cui contenuto verrà sommato ad una parte specifica dell'istruzione. Permette al programmatore una gestione più agevolata di cicli e rende possibile il meccanismo di ritorno da subroutine. Questo meccanismo verrà discusso e ne sarà più chiaro lo scopo una volta affrontato il capitolo 5.

- *Manuale Base di Programmazione*: cap. 2 (pag. 11–21); §4.1–4.3 (pag. 33–47); §6.2–6.4 (pag. 60–71); §7.1–7.3 (pag. 79–110); §7.10 (pag. 151).
- *Introduzione al Sistema ELEA*: Parte E, cap. 3 (pag. 146–149), §4.3 (pag. 153–155) e §9.2 (pag. 167–168).
- *Descrizione logica degli organi*: capp. 4–5 (pag. 19–37); cap. 8 (pag. 70–93).

Capitolo 4

Logica aritmetica

Prima di introdurre la logica della programmazione, l'instruction set di Olivetti e illustrare dei programmi da me sviluppati — argomenti trattati nel capitolo 5 — ritengo particolarmente interessante un focus su come vengano effettuate le operazioni aritmetiche, quale sia la codifica utilizzata e il perché, viste le sostanziali differenze con l'approccio di progettazione dei calcolatori recentemente adottato.

4.1 La rappresentazione numerica

Nei moderni sistemi di calcolo, la rappresentazione dell'informazione è generalmente conforme a standard comuni dell'industria — come la codifica ASCII o Unicode — che lasciano poco spazio a implementazioni alternative. Durante lo sviluppo di Elea non erano ancora ben delineate le specifiche di uno standard di rappresentazione, lasciando ai progettisti la possibilità di adottare codifiche che rispecchiassero necessità architetturali specifiche, soprattutto analizzando la questione dal punto di vista dell'aritmetica.

Per la rappresentazione numerica, ad oggi si opta per una codifica binaria pesata. Con una tale implementazione è possibile rappresentare valori su più cifre decimali mantenendo la codifica di ogni cifra (ad esempio tramite BCD) oppure salvare il dato in una stringa unica di bit: quest'ultima soluzione evita gli sprechi di combinazioni non utilizzate insiti nella rappresentazione cifra per cifra, e consente una logica di riporto uniforme indipendente dal numero di cifre coinvolte.

Prendiamo ora un caso d'esempio: effettuare una semplice somma, confrontando come la esegue un calcolatore generico e come viene eseguita in Elea. Inizialmente, entrambi ricevono in memoria una stringa di n caratteri — uno in ASCII Esteso, l'altro nella codifica in esame, funzionalmente analoga. La differenza emerge nella fase di calcolo: il primo deve tradurre da ASCII esteso a binario pesato, svolgere il calcolo in complemento a due e convertire il risultato, per essere stampato a schermo. Il secondo invece salta i passaggi di conversione, andando a svolgere i calcoli direttamente con la codifica ricevuta in ingresso.

Questa prima differenza non ha rilevanza — in negativo — per gli elaboratori moderni, vista la semplicità di tale conversione. Dove risiede quindi il vantaggio di una codifica diversa? La risposta è semplice: una questione di costi. Citando dal manuale, la codifica adottata "riduce al minimo il materiale necessario per realizzare le 'tavole' di somma e di prodotto"[1].

Per inciso, con "tavola" si intende un circuito combinatorio che implementa una tabella di verità, dato che non viene svolto nessun algoritmo di calcolo. Questa attenzione ai costi circuitali non è una peculiarità di Elea: l'assenza di uno standard dominante rendeva la scelta di una codifica tutt'altro che scontata anche fra i concorrenti dell'epoca, "il sistema binario puro non è generalmente usato nei calcolatori commerciali, nei quali si preferisce mantenere la rappresentazione decimale delle grandezze, pur facendo uso dei soli due simboli binari"[2]. A testimonianza del fatto che la scelta adottata nella progettazione di Elea non rappresenta un caso isolato.

Codifica del nastro perforato

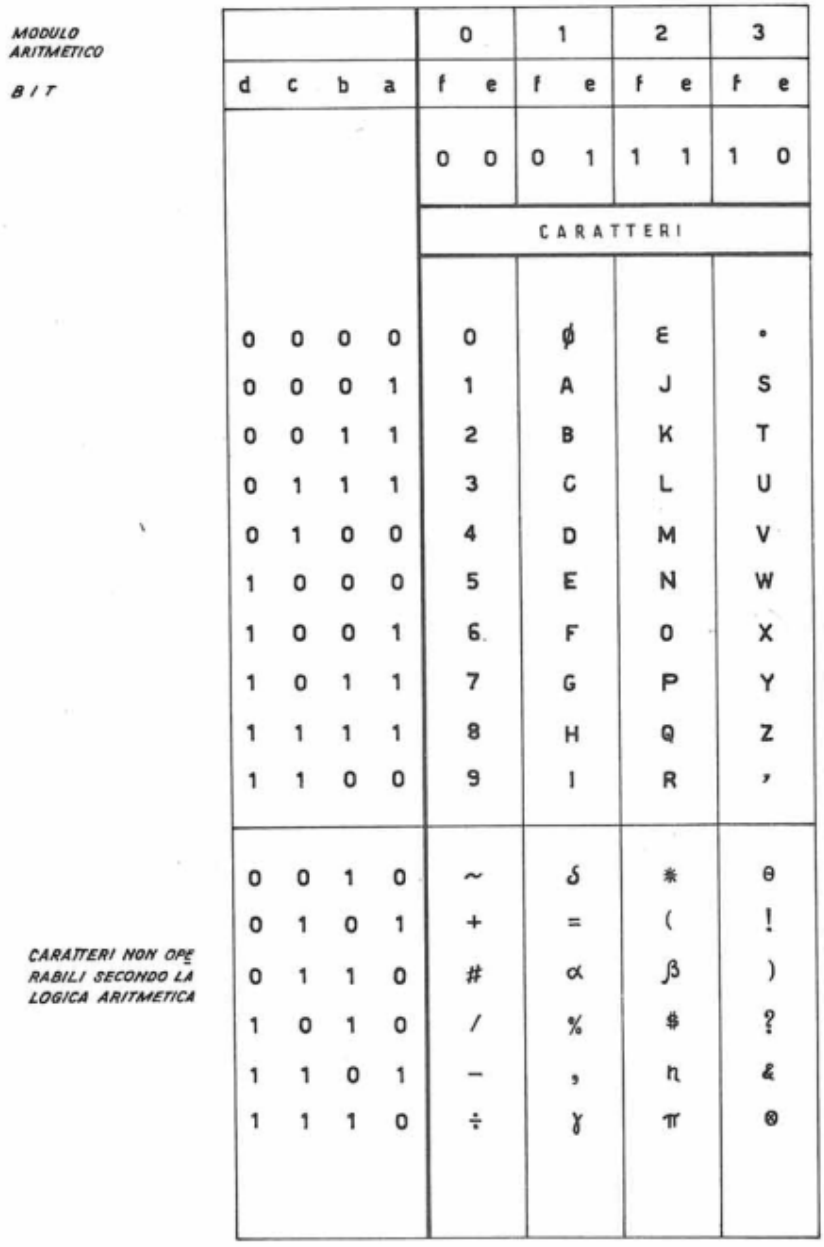
Per chiudere la questione conversioni, il nastro perforato, usato per lo scambio di dati con apparecchiature esterne, impiega una codifica diversa da quella interna di Elea. La ragione, stavolta, va cercata nel rapporto con il mondo esterno: l'introduzione di un elaboratore in un'azienda "non può prescindere dall'esistenza (presso la stessa azienda) di elaboratori meno potenti (ad esempio meccanografici) installati precedentemente", il che "provoca la coesistenza di diversi supporti fisici e la necessità per l'automa di essere in grado di utilizzarli tutti"[3].

La banda perforata può quindi essere letta sia direttamente dall'elaboratore, tramite il fotolettore — che, pur essendo un organo lento, è collegato direttamente alla macchina, rallentandola al proprio ritmo — sia fuori linea, tramite un apposito convertitore che la trascrive su nastro magnetico. In questo secondo caso l'elaboratore può scambiare informazioni con l'esterno "facendo esclusivo uso di nastri magnetici, a tutto vantaggio della velocità di lavoro"[4].

4.1.1 La codifica

La codifica utilizzata è rappresentata tramite sei bit, identificati dai simboli (divisi in due parti) "f, e" e "d, c, b, a". Le combinazioni della prima parte specificano a quale dei quattro insiemi di 16 simboli appartiene tale carattere — ad esempio, tutte le cifre hanno i bit $f, e = 00$. Sono gli stessi due bit già incontrati nel capitolo precedente per la selezione del modulo fisico di memoria: uno stesso campo del codice carattere che assume un significato diverso a seconda del contesto in cui il carattere compare — indirizzo o valore aritmetico. I quattro gruppi sono numerati

[1] Introduzione al Sistema ELEA, Parte A, pag. 6.
[2] Introduzione al Sistema ELEA, Parte A, pag. 4.
[3] Introduzione al Sistema ELEA, Parte E, §2.3, pag. 143.
[4] Introduzione al Sistema ELEA, Parte E, §10, pag. 170



MODULO ARITMETICO / BIT					0		1		2		3	
	d	c	b	a	f	e	f	e	f	e	f	e
					0	0	0	1	1	1	1	0
					CARATTERI							
	0	0	0	0	0		ϕ		Ε		•	
	0	0	0	1	1		A		J		S	
	0	0	1	1	2		B		K		T	
	0	1	1	1	3		C		L		U	
	0	1	0	0	4		D		M		V	
	1	0	0	0	5		E		N		W	
	1	0	0	1	6		F		O		X	
	1	0	1	1	7		G		P		Y	
	1	1	1	1	8		H		Q		Z	
	1	1	0	0	9		I		R		,	
	0	0	1	0	~		δ		*		θ	
	0	1	0	1	+		=		(		!	
CARATTERI NON OPERABILI SECONDO LA LOGICA ARITMETICA	0	1	1	0	#		α		β		)	
	1	0	1	0	/		%		$		?	
	1	1	0	1	−		,		η		&	
	1	1	1	0	÷		γ		π		⊗	

Figura 4.1: L'alfabeto dell'elaboratore con relativa codifica. Fonte: [7]

da 0 a 3, ma codificati nei bit f, e nell'ordine 00, 01, 11, 10: il gruppo 2, ad esempio, corrisponde a $f, e = 11$.

La seconda porzione dei bit, d, c, b, a, svolge invece un ruolo indipendente dal gruppo di appartenenza: determina se quella particolare configurazione sia *operabile* secondo la logica aritmetica. Come mostra la figura 4.1, sei delle sedici combinazioni possibili di d, c, b, a sono "non operabili secondo la logica aritmetica" — e lo sono allo stesso modo in ciascuno dei quattro gruppi, poiché il criterio dipende unicamente da questi quattro bit, non dal carattere di gruppo f, e. Sono cioè due filtri indipendenti applicati allo stesso carattere: uno ne stabilisce la famiglia, l'altro ne stabilisce l'idoneità al calcolo — una distinzione che permette di effettuare controlli rapidi sugli operandi non validi.

4.2 Realizzazione dell'aritmetica

Entriamo brevemente nel dettaglio di come viene gestita l'aritmetica, quanto sufficiente per giustificare la scelta della codifica. Innanzitutto, somma e prodotto non sono realizzate da due unità distinte, ma da un circuito singolo, generale. Esso implementa la seguente formula: $R \times M + A = C$ (dove R è il moltiplicatore, M l'operando di memoria, A l'accumulatore o un registro T, C il risultato), applicata cifra per cifra. L'implementazione di un circuito singolo, riutilizzando parte della componentistica per entrambe le operazioni, risulta quindi un significativo risparmio economico.

All'interno di tale circuito si sfrutta la simmetria "quinaria" del codice, derivata dalla presenza del bit d pesato a 5, i bit c, b, a infatti si ripetono identici ogni 5 cifre decimali. È questa simmetria che permette appunto di riutilizzare la circuiteria, in funzione della tavola utilizzata:

- nella tavola del prodotto, la simmetria dimezza il numero di configurazioni distinte necessarie per tre dei quattro bit d'uscita, rendendo la metà destra e quella sinistra della tavola uguali;
- nella tavola della somma, lo stesso principio la riduce a un quarto.

Può sorgere un dubbio, visto l'utilizzo della stessa circuiteria per entrambe le operazioni: come si comporta l'elaboratore quando è necessario effettuare solo una delle due? In realtà l'elaboratore non distingue i due casi: esegue sempre, indistintamente, sia la somma che il prodotto. Quel che succede è che, quando si vuole effettuare solo una somma, il registro R viene forzato automaticamente a 1, lasciando M inalterato. Non esiste un meccanismo hardware speculare per isolare il solo prodotto: è compito del programmatore azzerare preventivamente l'Accumulatore se non desidera che il suo contenuto si sommi al risultato della moltiplicazione.

Proprio perché il circuito appena descritto opera cifra per cifra su una codifica non interamente pesata, non può limitarsi a un'ordinaria somma binaria: quest'ultima presuppone infatti una rappresentazione posizionale pesata, che qui è presente solo

in parte. Basta un esempio per verificarlo, sommando in binario puro due semplici cifre decimali:

			f	e	d	c	b	a
		3	0	0	0	1	1	1
	+	1	0	0	0	0	0	1
(!)	=	5	0	0	1	0	0	0

Il risultato binario `001000`, decodificato secondo la codifica, corrisponde a **5**, non a 4. In caso di un riporto, esso viene sommato alla cifra successiva. Se la cifra successiva non è presente, si aggiunge una cifra al risultato quando si opera nell'accumulatore, altrimenti il riporto viene scartato.

Lo stesso problema si ripropone, in forma più articolata, quando l'operando non è una cifra ma una lettera: le operazioni aritmetiche sono infatti ammesse anche sui caratteri alfabetici, mentre quelle sui caratteri speciali non possono essere effettuate e producono un'indicazione di errore nell'unità aritmetica e logica. Vediamo dunque come il meccanismo appena descritto si estenda alle lettere.

Aritmetica su simboli

La differenza sostanziale rispetto al caso numerico è che un carattere alfabetico è identificato dai bit di gruppo, f, e, che verranno gestiti *indipendentemente* dal risultato della parte dettata da d, c, b, a, secondo due aritmetiche separate:

1. la parte numerica si somma normalmente — modulo 10 — tenendo conto del valore attribuito al simbolo;

2. la parte di gruppo si somma a sua volta, indipendentemente dall'esito della prima, secondo un'aritmetica modulo 4.

Per proporre un esempio che mostri tale meccanismo, la somma di `D` (gruppo 1, codifica di 4) con se stesso:

		f	e	d	c	b	a
	D	0	1	0	1	0	0
+	D	0	1	0	1	0	0
=	Q	1	1	1	1	1	1

La parte numerica risulta in $4+4 = 8$, senza riporto, così come la parte di gruppo, che dà $1+1 = 2$. A prescindere dalla presenza di un riporto numerico, le due somme procedono appunto per conto proprio. Cambiando gruppo, il carattere risultante è semplicemente la rappresentazione del valore numerico, in un gruppo diverso.

Un caso con riporto rivela invece una parte più delicata del meccanismo. Sommando infatti `D` a `I` (gruppo 1, valore 9) risulta:

			f	e	d	c	b	a
		D	0	1	0	1	0	0
	$+$	I	0	1	1	1	0	0
$(+1)$	$=$	L	1	1	0	1	1	1

La parte numerica, $4 + 9 = 13$, restituisce un riporto; la parte di gruppo rimane analoga all'esempio precedente ($1 + 1 = 2$), perché la sua somma non dipende in alcun modo dal riporto della parte numerica. Il risultato è quindi `L` (gruppo 2, valore 3), accompagnato da un secondo carattere che esprime il riporto numerico (che apparterrà al gruppo 0, quello delle cifre).

La divisione

Tra le operazioni aritmetiche proposte da Elea 9003, la mancanza che risalta più all'occhio è l'assenza della divisione. Le motivazioni sull'omettere tale operazione ricadono verosimilmente sia sul piano tecnologico che su quello commerciale. Da un punto di vista di complessità hardware, la codifica vista in questo capitolo rende la somma e la moltiplicazione — che, come visto, condividono lo stesso circuito — operazioni a basso costo circuitale, ma non offre lo stesso vantaggio alla divisione che richiederebbe a ogni passo un confronto di grandezza fra resto parziale e divisore. Sul piano commerciale, coerentemente con l'impiego di Elea discusso nel capitolo 1, la contabilità aziendale dell'epoca richiedeva prevalentemente somme, sottrazioni e moltiplicazioni; la divisione, quando necessaria, è delegata al programmatore — che può realizzarla in software per sottrazioni successive, come nella subroutine discussa nel capitolo 5.

4.2.1 La gestione del segno

Oltre alle operazioni su valori assoluti, è possibile gestire operandi segnati. Un numero dotato di segno occupa, in memoria, una posizione in più rispetto al suo valore, per ospitare appunto il proprio segno, nella cella di indirizzo più alta dell'operando.

Non esistendo un circuito di sottrazione dedicato, questa viene sempre eseguita come una somma: l'operando in memoria, quando necessario, viene complementato — in base 10, cifra per cifra — tramite un circuito complementatore dedicato, inserito sul canale interno in uscita dalla memoria, e poi sommato all'accumulatore o a un registo T. Il risultato di una tale somma può presentarsi in due forme distinte: in **vera grandezza**, cioè nella rappresentazione classica del valore, esattamente come le cifre di un numero non segnato; oppure in **complemento**. Quest'ultima forma è pensata per restare nell'accumulatore: quando un numero segnato viene trasferito in memoria, se si trova in complemento viene riportato automaticamente in vera grandezza. Se invece il numero non è segnato, il trasferimento avviene senza questa correzione: in memoria arriva il complemento, e l'informazione sul segno va perduta.

Quale delle due forme si ottenga, e se sia stato necessario svolgerne il complemento, non dipende dal solo segno degli operandi coinvolti: ogni operazione coinvolge infatti fino a quattro parametri di segno — quello dell'operazione richiesta, quello dell'operando in memoria, quello dell'operando nell'accumulatore e, se è in corso una moltiplicazione, quello del moltiplicatore.

In base al numero di segni "meno" complessivamente presenti in questi registri, si distinguono due casi:

- se **pari**, i segni negativi si annullano a vicenda: l'operazione procede come una semplice somma diretta, il cui risultato compare sempre in vera grandezza, senza che sia necessaria alcuna complementazione;
- se **dispari**, l'operando in memoria viene complementato e il suo segno invertito, prima ancora che venga eseguita l'operazione richiesta. Solo in questo caso l'esito dipende anche dalla presenza del riporto: se presente, viene scartato e il risultato compare comunque in vera grandezza; se assente, il risultato resta invece in complemento.

- *Manuale Base di Programmazione*: §1.3 (pag. 4–5); cap. 3 (pag. 23–32); §4.2 (pag. 37–45); §4.4 (pag. 48–52); §6.2 (pag. 60–63); §7.1 (pag. 79–92); §7.5 (pag. 121–125).
- *Introduzione al Sistema ELEA*: Parte A, capp. 1–2 (pag. 1–7); Parte E, §2.2–2.3 (pag. 140–144), §9.3 (pag. 168–169) e cap. 10 (pag. 170).
- *Descrizione logica degli organi*: §4.7 (pag. 25–26); §5.10 (pag. 33–36); cap. 7 (pag. 40–69).
- *Descrizione logico funzionale*: capp. 6–7 (pag. 39–55); cap. 11 (pag. 67–70).

Capitolo 5

L'assembler Olivetti

Come per la codifica, anche il linguaggio di programmazione che la macchina è in grado di interpretare non deriva da uno standard del settore. Nei manuali si presenta infatti come incipit il problema di dover scrivere un programma in linguaggio macchina, facilmente eseguibile dall'elaboratore ma difficilmente gestibile da un operatore umano. Per questo motivo è proposto un linguaggio simbolico che faciliti al programmatore la stesura di un programma, spostando l'onere della traduzione sull'elaboratore stesso — principio fondante dei linguaggi simbolici moderni come l'assembler in sintassi AT&T.

Il componente atto alla traduzione precedentemente citato è un decodificatore combinatorio a diodi che, in funzione del carattere F dell'istruzione appena letta — immagazzinato nell'apposito registro di funzione, il registro F appunto — attiva il segnale corrispondente fra i 64 possibili, avviando così l'esecuzione degli organi competenti per quel tipo di istruzione.

Le istruzioni sono presentate con un nome simbolico, come ad esempio `Y`, `AoM`, che ne descrivono il funzionamento generale, utili e comprensibili unicamente al programmatore, e tramite una codifica alfanumerica che racchiude informazioni più dettagliate sull'utilizzo dell'istruzione in un dato momento. Quest'ultima è il punto d'incontro tra il programmatore e l'elaboratore, è infatti l'unica codifica che l'elaboratore comprende e traduce. Tale nome simbolico rappresenta spesso un acronimo, che permette al programmatore una lettura dei programmi ulteriormente semplificata, ad esempio `AoM` diventa "Accumulatore Memoria", con la "o" a simboleggiare il conseguente azzeramento dell'accumulatore.

5.1 Il formato delle istruzioni

Ogni istruzione presenta otto caratteri, organizzati secondo uno schema comune che viene leggermente adattato al tipo di istruzione. I caratteri occupano le posizioni da p1 a p8 e si scrivono partendo da p8, a sinistra, a p1, a destra: il carattere di funzione, in p1, è quindi l'ultimo a destra, ma il primo a essere esaminato dalla macchina. La

configurazione di base comprende: lunghezza dell'operando (`LL`), indirizzo (`IIII`), registro modificatore (`Tm`) e carattere di funzione (`F`).

L'indirizzo `IIII` non individua, come si potrebbe supporre, il primo carattere dell'operando, bensì l'**ultimo**, quello meno significativo. Da lì la parola viene letta per indirizzi **decrescenti** verso le posizioni più significative: il carattere più significativo occupa dunque l'indirizzo di memoria più basso, mentre quello meno significativo l'indirizzo più alto — la stessa convenzione, concettualmente, del **Big-Endian** nei sistemi moderni. Questa convenzione vale per quasi la totalità delle istruzioni; fra quelle della tabella 5.1 fanno eccezione `CM`, `MS` e `LNa`, che scambiano dati con telescrivente, fotolettore e nastri, e la ricerca in avanti `RIa`: per esse `IIII` indica il **primo** carattere e si procede per indirizzi **crescenti**.

Il registro modificatore è utilizzato per la modifica, in tempo reale, dell'istruzione corrente. Come già accennato nella sezione dedicata ai registri T del capitolo 3, il contenuto del registro `Tm` in questione verrà sommato alla parte dell'istruzione relativa all'indirizzo (`IIII`), per agevolare ad esempio la gestione degli array. La somma avviene mentre l'istruzione viene letta, durante il suo passaggio attraverso l'unità aritmetica: l'istruzione registrata in memoria resta quindi invariata, e cambia soltanto la copia che viene eseguita.

Il carattere di funzione specifica il tipo di istruzione, ed è — esattamente come l'opcode in un'istruzione macchina moderna — una delle prime informazioni che vengono lette e gestite dall'elaboratore. Casistica particolare riguarda le istruzioni di salto: per esse il contenuto di F è sempre `"0"`, mentre è il carattere più a sinistra, in p8, detto **carattere di eventualità** (E), a specificare quale flag utilizzare per accertare se saltare o meno all'indirizzo specificato.

Per gli operandi di lunghezza non nota a priori esiste un'alternativa all'indicazione numerica in `LL`: il **carattere chiave**. Anziché specificare quanti caratteri elaborare, il programmatore indica nel campo `LL` quale carattere ne segnala la fine: un carattere preciso, scritto nella prima posizione del campo (p8) ponendo nella seconda (p7) `Q` o `P`, oppure, per gli operandi numerici, il primo carattere non numerico (escluso l'eventuale segno della parola, in `IIII`), ponendo nella seconda posizione `Z` o `Y`. L'operazione prosegue quindi finché non lo si incontra; con `Q` e `Z`, inoltre, l'indirizzo del carattere chiave viene registrato nel registro T9. Concettualmente è un meccanismo analogo a quello adottato per le stringhe in C, dove per conoscerne la lunghezza è necessario scorrere tutta la stringa finché non si incontra il carattere di fine stringa (`\0`).

Il formato appena descritto è, in tutti i casi visti, a lunghezza fissa: otto caratteri, sempre — caratteristica di molte architetture con instruction set RISC. In Elea sono però presenti anche istruzioni di ben altra complessità, come i trasferimenti diretti memoria-memoria (`PUM`+`MEM`/`±MM`/`CMM`, previa istruzione di preparazione) o le istruzioni di ricerca in memoria (`RIa`/`RIi`), che non si limitano a un singolo confronto ma esplorano autonomamente un intero campo di memoria, posizione dopo posizione, fino a individuare il carattere cercato — esattamente ciò che un'architettura RISC vieta per principio, riservando le operazioni ai soli registri e isolando l'accesso alla

memoria in istruzioni dedicate di load/store. Questo secondo criterio è molto più impattante rispetto alla lunghezza delle istruzioni, per classificare un instruction set, che quindi indicherei come CISC a lunghezza fissa.

Per orientarsi fra le istruzioni che verranno usate negli esempi di questo capitolo, la tabella 5.1 le raggruppa per famiglie funzionali e ne riporta la configurazione, con in evidenza il carattere che le identifica: quello di funzione F o, per i salti, quello di eventualità E. La tabella non rappresenta l'elenco completo dell'instruction set, ma comprende tutte le istruzioni utilizzate nei codici e prese d'esempio in questo testo. Per un'analisi dettagliata di tutte le istruzioni si rimanda alla bibliografia.

Il significato delle posizioni cambia infatti da un'istruzione all'altra. Oltre ai campi già visti, nelle configurazioni compaiono `L`, lunghezza di una sola cifra; `To`, registro T del secondo operando su cui si opera; `CCCCC`, costante di cinque cifre; `Ts`, registro T in cui il salto registra l'indirizzo del proprio carattere p8; `n`, unità a nastro.

Due convenzioni spiegano infine la presenza di `X` e `#`. Le posizioni indicate con `X` non sono esaminate dall'unità di governo e possono contenere un carattere qualsiasi; per il `#`, invece, vale una regola generale: "se in una istruzione una determinata posizione indica normalmente uno fra diversi organi della macchina, e non si vuole richiamare nessun organo di quel tipo, in essa si deve registrare un # (diesis)"[1]. Per questo `#` compare in `Tm` quando l'indirizzo non va modificato e in `Ts` quando non occorre ricordare da dove si è saltati. Nelle istruzioni sulle costanti (`CT`, `+CT`, `-CT`), invece, il `#` in p8 indica una posizione non utilizzata.

Tabella 5.1: Un sottoinsieme delle istruzioni

Istruzione	**Configurazione** p8 p7 p6 p5 p4 p3 p2 p1	**Funzione**
Accumulatore		
MA	L L I I I I T_m F	Copia memoria → accumulatore
AoM	L L I I I I T_m B	Copia accumulatore → memoria, con azzeramento
+MA	L L I I I I T_m C	Somma un operando di memoria all'accumulatore
-MA	L L I I I I T_m H	Sottrae un operando di memoria dall'accumulatore
CMA	L L I I I I T_m E	Confronto memoria-accumulatore
Registri T e moltiplicazione		
MT	L T_o I I I I T_m X	Copia memoria → registro T
TM	L T_o I I I I T_m Y	Copia registro T → memoria
ToM	L T_o I I I I T_m T	TM con azzeramento

[1] Manuale Base di Programmazione, §6.3, pag. 66.

Istruzione	**Configurazione** p8 p7 p6 p5 p4 p3 p2 p1	**Funzione**
CT	# C C C C C T_o 9	Carica una costante in un registro T
+CT	# C C C C C T_o +	Incrementa un registro T di una costante
-CT	# C C C C C T_o 4	Decrementa un registro T di una costante
Y	L L I I I I T_m 6	Carica il moltiplicatore in T0
+X	L L I I I I T_m =	Moltiplica un operando di memoria per i registri T, sommando il prodotto all'accumulatore
Memoria-memoria e ricerca		
PUM	L L I I I I T_m ÷	Prepara l'uscita da memoria di un operando per l'istruzione seguente
MEM	L L I I I I T_m 1	Copia memoria → memoria
+MM	L L I I I I T_m 3	Somma due operandi di memoria, con risultato in memoria
-MM	L L I I I I T_m 8	Sottrae due operandi di memoria, con risultato in memoria
CMM	L L I I I I T_m /	Confronto memoria-memoria
RIa	R C I I I I T_m 2	Ricerca in memoria il carattere R, confrontando solo i bit indicati da C, per indirizzi crescenti; ne registra l'indirizzo in T9
RIi	R C I I I I T_m 7	Come RIa, ma per indirizzi decrescenti
Ingresso e uscita		
CM	L L I I I I T_m %	Legge un operando da telescrivente o fotolettore
MS	L L I I I I T_m ∅	Scrive un operando sulla telescrivente
LNa	X n I I I I T_m *	Legge un blocco da nastro magnetico, in avanti
Salti (F è sempre 0: in evidenza il carattere di eventualità E)		
SI	) T_s I I I I T_m 0	Salto incondizionato
SC=	X T_s I I I I T_m 0	Salta se uguale
SC>	S T_s I I I I T_m 0	Salta se maggiore
SR=	θ T_s I I I I T_m 0	Salta se uguale a zero
SR≠	. T_s I I I I T_m 0	Salta se diverso da zero
SE	β T_s I I I I T_m 0	Salta su errore
SEN	M T_s I I I I T_m 0	Salta su errore di un'unità a nastro magnetico
S2P	⊠ T_s I I I I T_m 0	Avvia il 2° programma
S3P	, T_s I I I I T_m 0	Avvia il 3° programma
SUO	% U I I I I T_m 0	Salta se l'unità in linea U è occupata
STOP	0 X X X X X # 0	Termina il programma

5.2 Organizzazione della programmazione

Subroutine

Per effettuare chiamate a funzione, indipendenti dal programma chiamante, ci si appoggia principalmente all'istruzione di salto incondizionato `SI`, sfruttando in particolare due dei suoi campi, posizionati in punti distinti dell'istruzione e tra loro indipendenti: il registro `Ts`, il secondo carattere, dedicato a conservare l'indirizzo di ritorno e il registro di modifica `Tm`, collocato invece subito prima del carattere di funzione.

Per mantenere salvato il punto da cui si salta — per poterci tornare una volta terminata una subroutine — si può indicare nell'istruzione di salto utilizzata un registro `Ts` dedicato: in esso viene registrato l'indirizzo del carattere p8 dell'istruzione di salto (se la condizione di salto è rispettata). Per quanto riguarda il ritorno è sufficiente utilizzare una `SI` specificando come indirizzo `0015` e indicare nel campo relativo al registro di modifica il `Ts` precedentemente utilizzato — è quindi consigliabile utilizzare lo stesso registro per tutte le chiamate a funzione. Questa configurazione verrà utilizzata negli esempi di questo capitolo con `T9` come registro di modifica. Il valore `0015` riporta all'istruzione che segue la chiamata: da p8 servono 7 posizioni per arrivare al carattere F, il cui indirizzo è quello dell'istruzione di salto, e altre 8 per passare all'istruzione successiva. Se ad esempio la chiamata si trova all'indirizzo 0680, `T9` riceve 0673 e il ritorno salta a 0673 + 0015 = 0688.

Un limite di questo meccanismo, se confrontato con un'architettura moderna, deriva dall'assenza di uno stack. Una `call` x86 spinge l'indirizzo di ritorno su una struttura dati "LIFO", permettendo chiamate annidate o ricorsive senza che il programmatore debba occuparsene esplicitamente: ogni livello di annidamento ottiene automaticamente il proprio spazio. In Elea, invece, ogni subroutine richiede che il programmatore scelga staticamente un registro `Ts` per il proprio indirizzo di ritorno: se ad esempio la stessa subroutine viene richiamata per ricorsione, il secondo indirizzo di ritorno sovrascrive il primo, di fatto perdendolo.

Multiprogrammazione

Lo scopo di questa sezione, visto che la simultaneità operativa come concetto è già stata trattata, è quello di proporre come sfruttare sequenze di programmi concorrenti nel sistema Elea.

Per capire come sia possibile far procedere più sequenze di programma in parallelo, occorre prima richiamare come si articola, nel tempo, l'esecuzione di una singola istruzione.

Ogni istruzione si svolge in due fasi distinte. La prima, detta *fase preparatoria* (α), ha una durata fissa di nove periodi di cifra e impegna sempre il canale interno, qualunque sia il tipo di istruzione. La seconda, la *fase esecutiva* (β), ha invece una durata variabile che dipende dalla funzione specifica svolta dall'istruzione; il suo primo periodo di cifra, tuttavia, impegna anch'esso il canale interno, prima ancora

che l'esecuzione vera e propria — che potrà interessare l'uno o l'altro canale, o entrambi — abbia effettivamente inizio.

Poiché un periodo di cifra corrisponde a 10 μs, ne segue che ogni istruzione, ancora prima che sia possibile stabilire quale canale la sua esecuzione richiederà, impegna comunque il canale interno per dieci periodi di cifra consecutivi, cioè 100 μs.

Da questa struttura in due fasi si possono classificare le istruzioni in cinque tipologie, a seconda degli organi coinvolti nella fase esecutiva:

1. istruzioni che impegnano il solo canale interno (ad esempio `MA`, le operazioni aritmetiche su registri T);
2. istruzioni che impegnano il canale esterno e il governo delle unità a nastro magnetico (le letture e scritture su nastro);
3. istruzioni che impegnano sia il canale interno sia quello esterno (i trasferimenti memoria-memoria, `PUM`+`MEM`/`±MM`/`CMM`);
4. istruzioni che impegnano canale interno, canale esterno e GUN contemporaneamente;
5. istruzioni che impegnano il solo governo delle unità a nastro.

Due istruzioni possono procedere in parallelo solo se appartengono a due tipi che non contengono lo stesso organo. Nel dettaglio, un'istruzione del tipo 1 può sovrapporsi ad una del tipo 2 o 5; due istruzioni del tipo 3 invece non possono mai procedere in parallelo, perché ciascuna utilizza entrambi i canali.

Pattern a doppio buffer

Come precedentemente accennato, è compito del programmatore ottimizzare il più possibile la simultaneità operativa, e questo meccanismo è possibile se si sceglie accuratamente per ogni programma i tipi delle istruzioni e in quali quantità inserirle.

Come consigliato dal manuale, l'utilizzo ottimale risiede nell'impiegare il 1° programma per elaborare i dati mentre il 2° si occupa dell'introduzione e dell'estrazione degli stessi dal nastro — questo è inoltre lo schema che propongo negli esempi di codice del seguito. Per far entrare in conflitto il meno possibile l'elaborazione e il trasferimento, i dati non devono risiedere nella stessa zona di memoria contemporaneamente. Si utilizzano quindi due zone diverse, in alternanza: Z_i e Z'_i. Mentre il 2° programma usa una zona, il 1° elabora i dati presenti nell'altra in contemporanea, è questo possibile grazie alla suddivisione della memoria in matrice pari e matrice dispari descritta nel capitolo 3.

Una volta terminato il proprio compito, i due programmi si scambiano le zone in cui operano sincronizzandosi tramite `S2P`, è quindi inevitabile che uno dei due programmi aspetti l'altro per utilizzare l'altra zona di memoria.

5.3 Programmi d'esempio

Avendo introdotto i meccanismi principali da sfruttare durante la stesura di un programma nel sistema Elea 9003, propongo alcuni esempi che ne mostrino un utilizzo concreto.

Mi preme fare una premessa, prima di illustrare i codici. Vista la situazione in cui versa l'esemplare di Bibbiena, non è attualmente possibile eseguire questi programmi sull'elaboratore. Per diminuire il più possibile la probabilità che contengano errori, sono stati passati in rassegna anche da tecnici che hanno operato con Elea quando era ancora in grado di eseguire del codice.

5.3.1 Divisione

Il programma proposto di seguito implementa l'operazione di divisione per sottrazioni successive, sfruttando operandi non segnati su 8 cifre. Esso è pensato come subroutine, richiamabile da programmi arbitrariamente complessi, con l'unica limitazione di attenersi allo standard d'utilizzo precedentemente proposto e che venga inserito nelle locazioni di memoria corrette.

Inizialmente vengono letti dividendo (N) e divisore (D) come input dalla telescrivente tramite le due istruzioni di `CM`, per poi verificare che il divisore non sia zero; Per fare questo, l'accumulatore viene azzerato trasferendovi gli zeri contenuti in Z, e il suo contenuto confrontato con D tramite `CMA`. Tramite `SC=` si utilizza il risultato del confronto per eventualmente saltare al ramo d'errore.

Una volta entrati nel ciclo che effettua la divisione, ad ogni iterazione viene caricato il dividendo corrente nell'accumulatore e confrontato con il divisore: in caso il divisore risulti maggiore (`SC>`), il programma può terminare, con il resto già contenuto in N. In caso contrario, il divisore viene sottratto dall'accumulatore (`-MA`) e il risultato viene trasferito in memoria tramite `AoM` che ne azzererà il contenuto.

Il calcolo del quoziente si basa sull'incremento di un registro T tramite la sequenza `MT`/`+CT`/`TM`: il valore viene prelevato dalla memoria, incrementato nel registro, e riportato in memoria.

Il ciclo si chiude con un salto incondizionato (`SI`) che riporta all'inizio dell'iterazione. Una volta terminata la divisione, viene sempre utilizzata la `SI`, con la differenza che viene utilizzata per effettuare un ritorno da subroutine (`)# 0015 90`), dove si presume che l'indirizzo del chiamante sia salvato in `T9` secondo la convenzione esplicitata in precedenza.

I risultati sono ritornati implicitamente, il quoziente in Q e il resto nella locazione di memoria dove originariamente era proposto il dividendo N.

Tabella 5.2: Divisione (08 cifre)

Indirizzo	**Istruzione**	**Configurazione**	**Commento**
Dati da caricare in memoria			
0303–0310	(letto da CM)		Dividendo N
0313–0320	(letto da CM)		Divisore D
0323–0330	00000000		Quoziente Q
0333–0340	00000000		Costante Z = 0
0347–0350	DIV0		Messaggio d'errore
Programma			
0360	CM	08 0303 #%	Legge N
0368	CM	08 0313 #%	Legge D
0376	MA	08 0340 #F	Acc = Z
0384	CMA	08 0320 #E	Confronta D e accumulatore
0392	SC=	X# 0480 #0	D == 0 ? errore, salta a (0480)
0400	MA	08 0310 #F	*(LOOP)* Acc = N
0408	CMA	08 0320 #E	Confronta D e accumulatore
0416	SC>	S# 0472 #0	D > N ? fine (0472)
0424	-MA	08 0320 #H	Acc = N − D
0432	AoM	08 0310 #B	N = Acc; Acc = 0
0440	MT	81 0330 #X	T1 = Q
0448	+CT	#0 0001 1+	T1++
0456	TM	81 0330 #Y	Q = T1
0464	SI	)# 0400 #0	Torna al ciclo (0400)
0472	SI	)# 0015 90	*(FINE)* Ritorno (T9+0015)
0480	MS	04 0347 #∅	*(ERRORE)* Stampa "DIV0"
0488	STOP	O# #### #0	Fine per Errore

5.3.2 Calcolo della media

Il seguente programma calcola la media aritmetica di un vettore V di 5 elementi, presi in input dalla telescrivente. Esso sfrutta il meccanismo di modifica a runtime delle istruzioni tramite l'ausilio dei registri `Tm` per la gestione dell'array e utilizza il programma della divisione precedentemente illustrato come subroutine. Il ciclo di lettura e somma utilizza il registro T1 come indice: ad ogni iterazione T1 viene incrementato di 10 (la spaziatura fra un elemento del vettore e il successivo) e sommato all'indirizzo base tramite il campo `Tm` dell'istruzione, così da puntare progressivamente a V0, V1, ..., V4. Il registro T2 funge invece da contatore delle iterazioni: il suo decremento (`-CT`) imposta il flag di risultato che l'istruzione `SR≠` interroga per decidere se ripetere il ciclo.

Tabella 5.3: Media di 5 elementi

Indirizzo	**Istruzione**	**Configurazione**	**Commento**
Dati da caricare in memoria			
0313–0320	`00000005`		Divisore D, (# elementi)
0503–0510	(letto da `CM`)		V0
0513–0520	(letto da `CM`)		V1
0523–0530	(letto da `CM`)		V2
0533–0540	(letto da `CM`)		V3
0543–0550	(letto da `CM`)		V4
0553–0560	`00000000`		Media M
Programma			
0600	`CT`	`#0 0000 19`	T1 = 0 (indice del vettore)
0608	`CT`	`#0 0005 29`	T2 = 5 (contatore iterazioni)
0616	`MA`	`08 0340 #F`	Acc = 0
0624	`CM`	`08 0503 1%`	Legge V[i]
0632	`+MA`	`08 0510 1C`	Acc + = V[i] (0510+T1)
0640	`+CT`	`#0 0010 1+`	T1 + = 10
0648	`-CT`	`#0 0001 24`	T2−−
0656	`SR≠`	`.# 0624 #0`	T2 ! = 0 ? salta a (0624)
0664	`AoM`	`08 0310 #B`	N = Acc; Acc = 0
0672	`AoM`	`08 0330 #B`	Q = Acc
0680	`SI`	`)9 0376 #0`	Call Divisione
0688	`MA`	`08 0330 #F`	*(RITORNO)* Acc = Q
0696	`AoM`	`08 0560 #B`	M = Acc
0704	`MS`	`08 0553 #∅`	Stampa la media
0712	`STOP`	`0# #### #0`	Fine programma

5.3.3 Calcolo di una fattura

Il programma seguente riproduce, in forma ridotta, una probabile elaborazione tipica dell'impiego commerciale di Elea: il calcolo del totale di una fattura. Sul nastro magnetico sono registrate le righe della fattura, ciascuna in un proprio blocco, contenente: la quantità e il prezzo unitario di un articolo, in 4 cifre; il programma calcola la somma degli importi, cioè dei prodotti quantità × prezzo, e la stampa sulla telescrivente. L'istruzione `LNa` trasferisce in memoria l'intero blocco, compresi i due caratteri α che lo delimitano. Il calcolo sfrutta la forma della moltiplicazione vista nel capitolo 4: `Y` carica la quantità nei registri T a partire da T0, e `+X` la moltiplica per il prezzo, sommando il prodotto al contenuto dell'accumulatore. Il 1° programma esegue i calcoli, mentre le letture sono affidate al 2°, con due routine: lettura in A e lettura in B. Poiché `S2P` non viene eseguita finché il 2° programma non è giunto al proprio `STOP`, ogni `S2P` fa anche da punto di sincronizzazione: quando il 1° programma avvia la lettura in B, quella in A è certamente conclusa, e il calcolo sulla riga in A si sovrappone alla lettura di B; allo stesso modo, il calcolo sulla riga in B si sovrappone alla lettura della riga successiva in A.

- *Manuale Base di Programmazione*: capp. 6–10 (pag. 59–230).
- *Introduzione al Sistema ELEA*: Parte E, capp. 4–7 (pag. 150–165) e cap. 9 (pag. 167–169).
- *Descrizione logica degli organi*: capp. 1–3 (pag. 1–18); cap. 9 (pag. 94–99); cap. 12 (pag. 113–119).
- *Descrizione logico funzionale*: cap. 15 (pag. 109–117); cap. 19 (pag. 153–163); cap. 22 (pag. 195–206).
- *La memoria a nastri magnetici*: Parte A, §1.2 (pag. 2–3) e cap. 3 (pag. 9–15).

Tabella 5.4: Totale di una fattura

Indirizzo	**Istruzione**	**Configurazione**	**Commento**
Dati da caricare in memoria			
0800–0809	(letto da LNa)		Zona A: α, QA (0801–0804), PA (0805–0808), α
0810–0819	(letto da LNa)		Zona B: α, QB (0811–0814), PB (0815–0818), α
0821–0830	0000000000		Costante Z = 0
0841–0850	0000000000		Totale TOT
1° programma			
0900	CT	#0 0003 39	T3 = 3 (coppie di righe)
0908	MA	10 0830 #F	Acc = 0
0916	S2P	☒# 1200 #0	Avvia lettura A
0924	S2P	☒# 1216 #0	*(CICLO)* Attende A; avvia lettura B
0932	Y	04 0804 #6	T0 = QA
0940	+X	04 0808 #=	Acc + = PA × T0
0948	-CT	#0 0001 34	T3−−
0956	SR=	θ# 0996 #0	T3 == 0 ? salta a (0996)
0964	S2P	☒# 1200 #0	Attende B; avvia lettura A
0972	Y	04 0814 #6	T0 = QB
0980	+X	04 0818 #=	Acc + = PB × T0
0988	SI	)# 0924 #0	Torna al ciclo (0924)
0996	S2P	☒# 1232 #0	*(FINE)* Attende B
1004	Y	04 0814 #6	T0 = QB
1012	+X	04 0818 #=	Acc + = PB × T0
1020	AoM	10 0850 #B	TOT = Acc; Acc = 0
1028	MS	10 0841 #∅	Stampa il totale
1036	STOP	O# #### #0	Fine 1° programma
2° programma			
1200	LNa	X1 0800 #*	Lettura A (0800→0809)
1208	STOP	O# #### #0	Fine routine
1216	LNa	X1 0810 #*	Lettura B (0810→0819)
1224	STOP	O# #### #0	Fine routine
1232	STOP	O# #### #0	Wait

Considerazioni finali

Come già accennato, è stato possibile scrivere questo testo grazie a molti fattori. Primo su tutti, il fatto che ad oggi l'Elaboratore Elettronico Aritmetico Elea 9003/02 sia in condizioni di semi-operatività, il che mi ha permesso di toccare con mano la maggior parte delle componenti descritte finora. Vedere la passione di volontari che cercano al meglio delle loro possibilità di far sopravvivere una macchina di questo calibro mi ha mosso più di tutti nella stesura di questo elaborato.

Sollevo quindi un appello, per quanto la mia voce possa esser fioca nel panorama accademico, sullo stato dell'elaboratore. Parlando con tutta franchezza, non sopravviverà nelle condizioni in cui riversa per molti anni, stiamo permettendo che una delle eredità più importanti del nostro paese, per quanto concerne la storia del nostro settore, venga dimenticata.

Ringrazio in primis il mio relatore, Giuseppe Lettieri, per avermi appoggiato nel progetto, per avermi dato una voce. Ringrazio i tecnici, membri dell'associazione "Elea ETS", con un ringraziamento particolare per Wladimir Zaniewski, che mi ha accompagnato durante la stesura dandomi consigli importanti e revisionando i miei codici.

Ringrazio i miei genitori, Andrea e Francesca, per il supporto emotivo dimostrato negli anni, per avermi appoggiato nelle scelte, per aver creduto in me.

Ringrazio i miei colleghi, per esser stati così determinanti nel proseguimento del mio percorso di studi. Posso infatti esprimere con certezza un fatto: non sarei mai riuscito a laurearmi senza il loro aiuto.

Dedico infine questa tesi a Claudio, per il suo supporto incondizionato.

Bibliografia

[1] Maurizio Gazzarri. *Elea 9003. Storia del primo calcolatore elettronico italiano.* Edizioni di Comunità, 2021.

[2] Laboratorio Ricerche Elettroniche Olivetti. *Elea 9003: La memoria a nastri magnetici.* Olivetti, Borgolombardo, Milano, ottobre 1960.

[3] Laboratorio Ricerche Elettroniche Olivetti. *Introduzione al Sistema Elea: Cenni Propedeutici.* Olivetti, Borgolombardo, Milano, aprile 1960.

[4] Laboratorio Ricerche Elettroniche Olivetti. *L'Unità Centrale Elea 9003: Descrizione logica degli organi.* Olivetti, Borgolombardo, Milano, aprile 1960.

[5] Laboratorio Ricerche Elettroniche Olivetti. *L'Unità Centrale Elea 9003: Descrizione logico funzionale.* Olivetti – Servizio Tecnico Elettronico Esterno, Borgolombardo, Milano, ottobre 1962.

[6] Elisabetta Mori. Olivetti elea 9003 storia e design del primo computer commerciale italiano. Master's thesis, Università degli Studi di Firenze, novembre 2012.

[7] Olivetti – Divisione Elettronica. *Elea 9003: Manuale base di programmazione*, 1960.